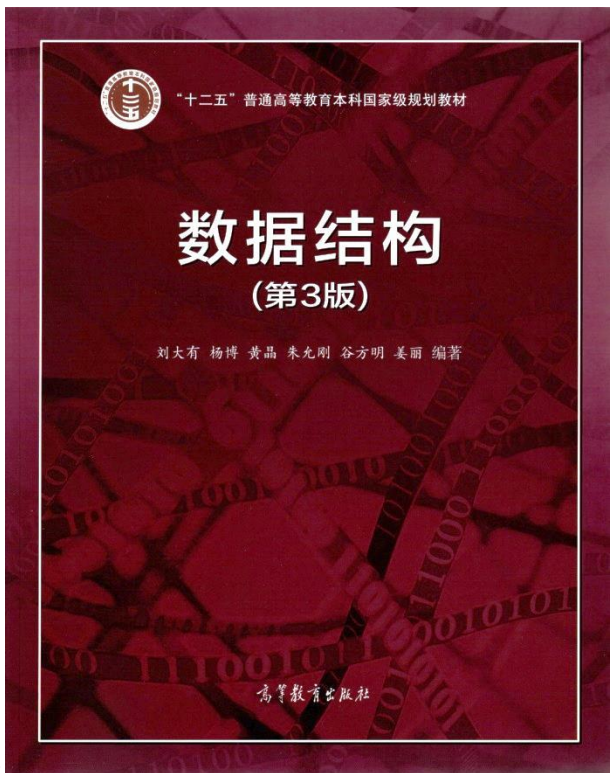


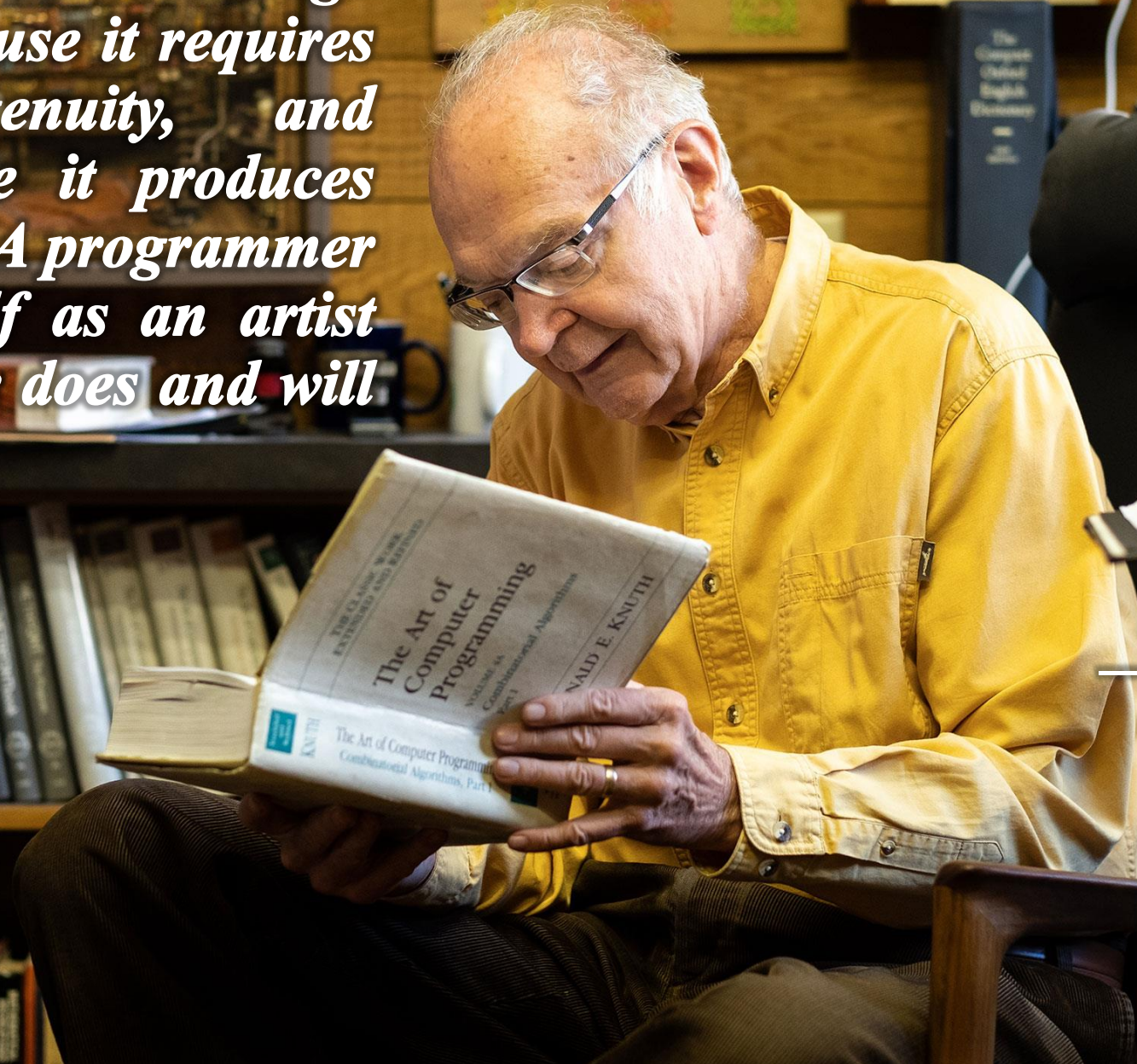


字符串匹配

- 模式匹配基本概念
- 朴素模式匹配
- KMP算法
- KMP算法的改进

数据之法
结构之美
算法之道

Computer programming is an art, because it applies knowledge to the world, because it requires skill and ingenuity, and especially because it produces objects of beauty. A programmer who views himself as an artist will enjoy what he does and will do it better.

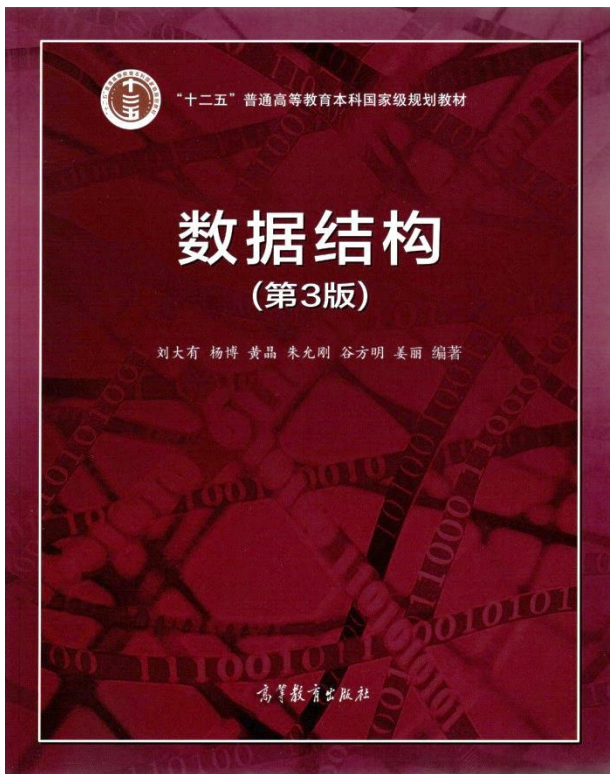


——Donald Knuth
图灵奖获得者
斯坦福大学教授
美国科学院院士
美国工程院院士



慕课自学内容（必看，计入期末成绩）

自学内容	视频时长	主讲人
字符串的定义与字符串类	7分56秒	杨博教授



字符串模式匹配

- 模式匹配基本概念
- 朴素模式匹配算法
- KMP算法
- KMP算法的改进

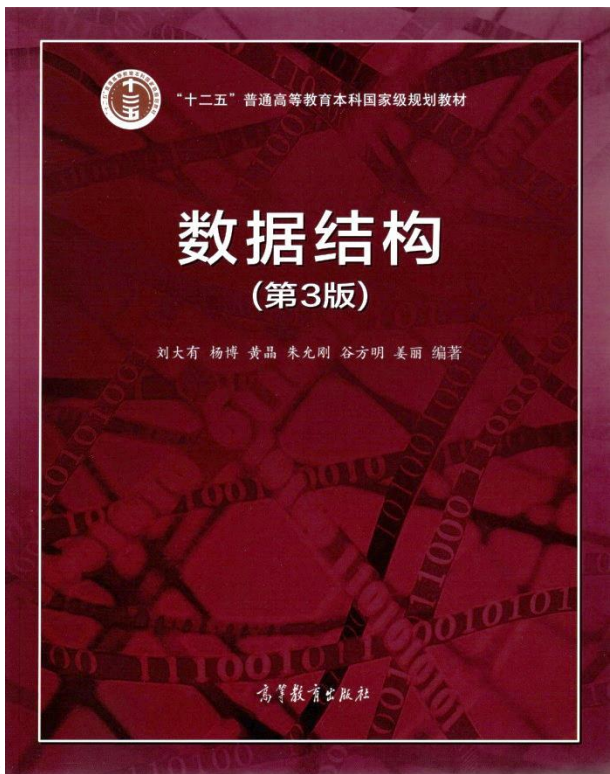
数据之法
结构之美
算法之道

字符串模式匹配问题定义

- 在目标串中寻找模式串出现的首位置。
- 给定两个字符串 S 和 P ，目标串 S 有 n 个字符，模式串 P 有 m 个字符， $m \leq n$ 。判断 P 是否为 S 的子串，如果是则返回 P 的第一个字符在 S 中的首位置；如果不是则返回-1。【大厂面试题 [LeetCode28](#)】

例： $S = \text{"abaabab"}$, $P = \text{"abab"}$

- 模式匹配应用：文本编辑器中常用的“查找”、“替换”



字符串模式匹配

- 模式匹配基本概念
- **朴素模式匹配算法**
- KMP算法
- KMP算法的改进

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

朴素模式匹配算法（Brute Force算法，暴力算法）

A

	0	1	2	3	4	5	6	7
<i>S</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>
<i>P</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>				
	0	1	2	3				

第 0 趟匹配（看 *P* 是否在 *S* 的第 0 个位置）

朴素模式匹配算法 (Brute Force算法, 暴力算法)

A

	0	1	2	3	4	5	6	7
<i>S</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>
<i>P</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>				
	0	1	2	3				

第 0 趟匹配 (看 *P* 是否在 *S* 的第 0 个位置)

第 1 趟匹配 (看 *P* 是否在 *S* 的第 1 个位置)

朴素模式匹配算法 (Brute Force算法, 暴力算法)

A

	0	1	2	3	4	5	6	7
<i>S</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>
<i>P</i>								
			<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>		
	0	1	2	3				

第 0 趟匹配 (看 *P* 是否在 *S* 的第 0 个位置)

第 1 趟匹配 (看 *P* 是否在 *S* 的第 1 个位置)

第 2 趟匹配 (看 *P* 是否在 *S* 的第 2 个位置)

...

朴素模式匹配算法 (Brute Force算法, 暴力算法)

A

	0	1	2	3	4	5	6	7								
<i>S</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>								
<i>P</i>	<table><tr><td><i>a</i></td><td><i>b</i></td><td><i>a</i></td><td><i>b</i></td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>				<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	0	1	2	3				
<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>													
0	1	2	3													

第 0 趟匹配 (看 *P* 是否在 *S* 的第 0 个位置)

第 1 趟匹配 (看 *P* 是否在 *S* 的第 1 个位置)

第 2 趟匹配 (看 *P* 是否在 *S* 的第 2 个位置)

...

朴素模式匹配算法 (Brute Force算法, 暴力算法)

A

	0	1	2	3	4	5	6	7
<i>S</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>
<i>P</i>					<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>
					0	1	2	3

第 0 趟匹配 (看 *P* 是否在 *S* 的第 0 个位置)

第 1 趟匹配 (看 *P* 是否在 *S* 的第 1 个位置)

第 2 趟匹配 (看 *P* 是否在 *S* 的第 2 个位置)

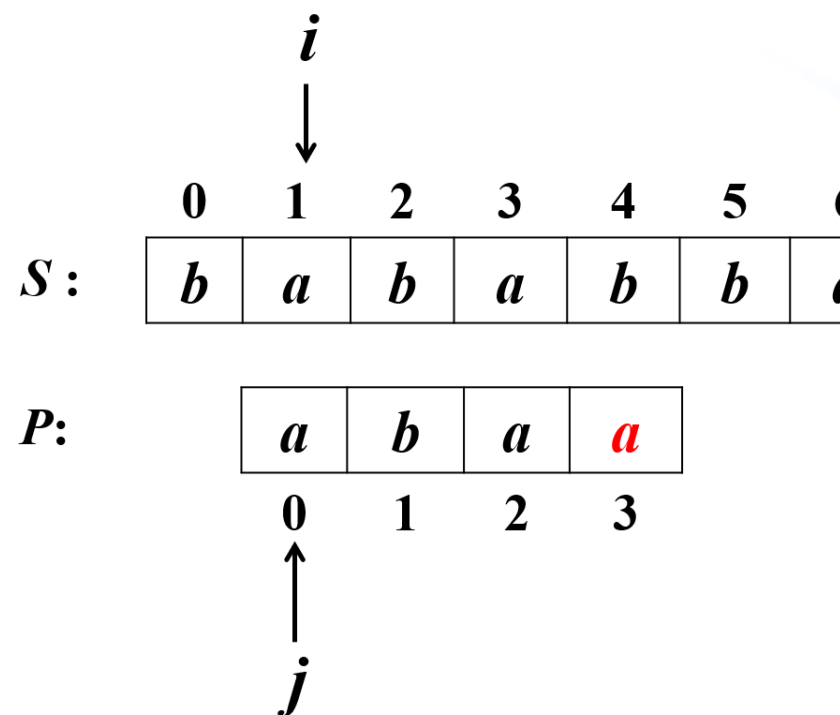
...

朴素模式匹配算法

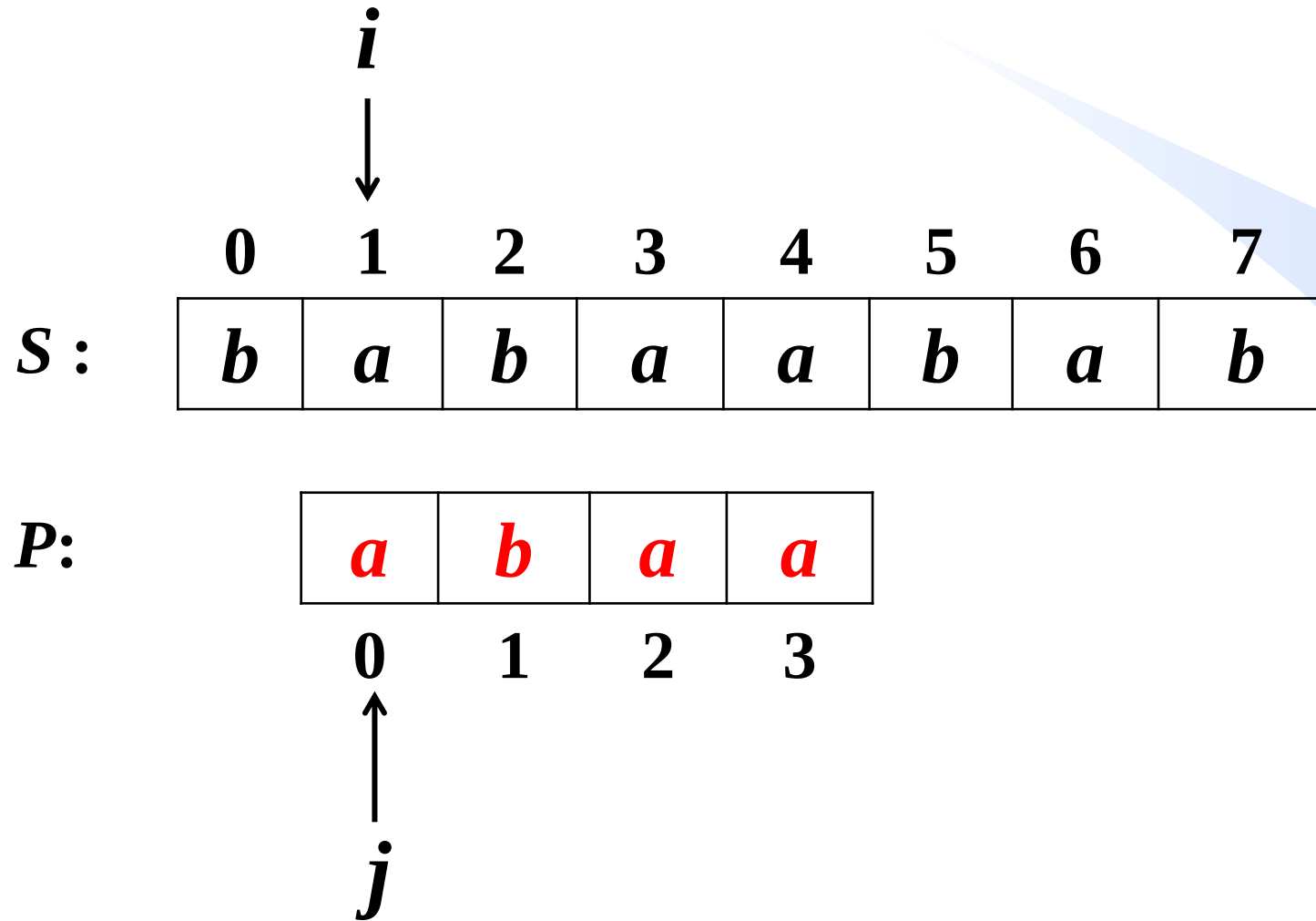
```

int StringMatching(char *s, char *p){ //返回p在s中的位置
    int n=strlen(s), m=strlen(p); //n为s的长度, m为p的长度
    int i = 0; //通过i扫描s
    while(i<=n-m){ //看p是否在s的第i个位置, 从s[i]开始与p逐字符比对
        int j = 0; //通过j扫描p
        while(j<m && s[i+j]==p[j]) {
            i++; j++;
        }
        if(j==m) return i-m; // 匹配成功
        .....未完待续
    }
}

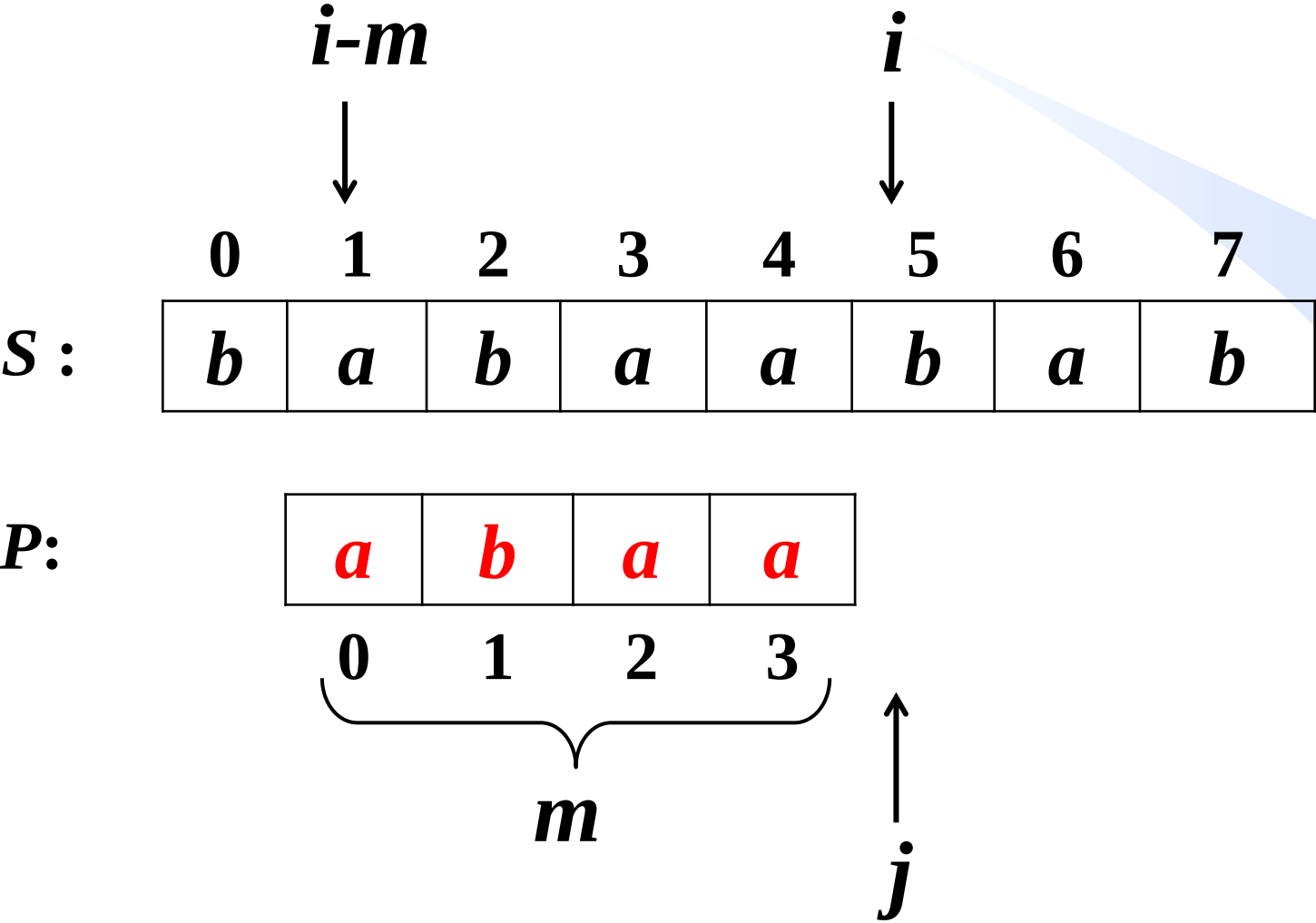
```



匹配成功的情况



匹配成功的情况

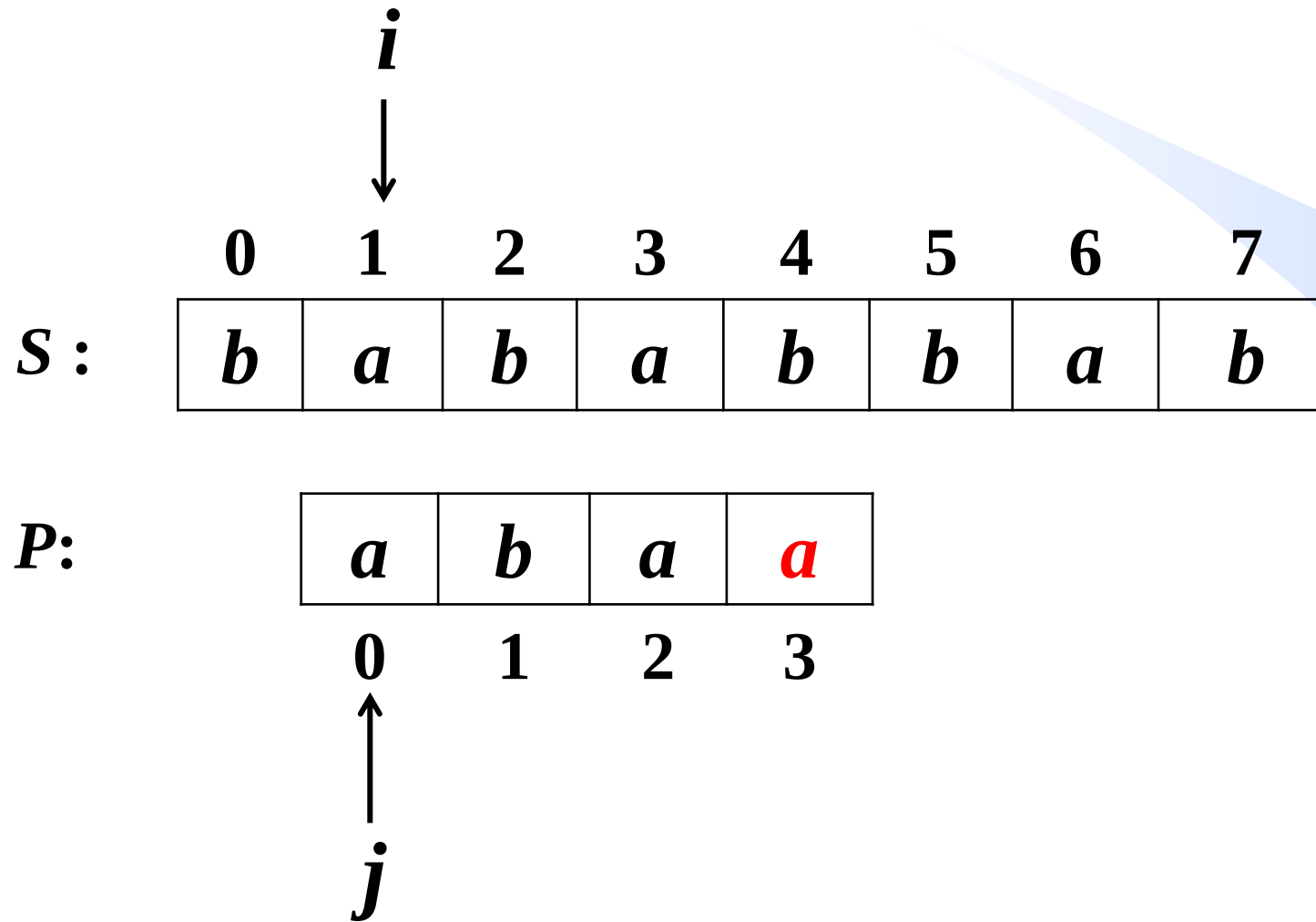


朴素模式匹配算法

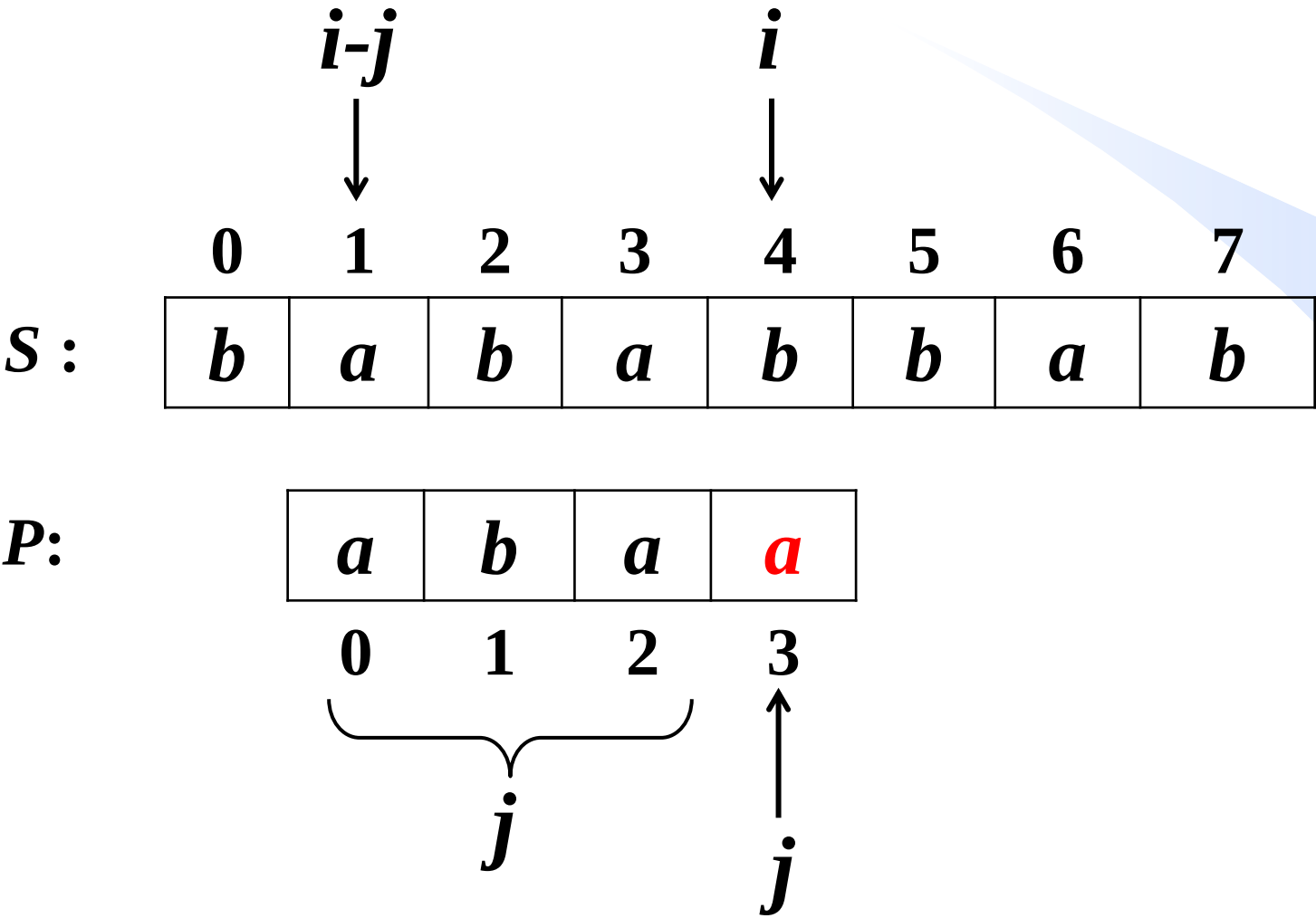
```
int StringMatching(char *s, char *p){ //返回p在s中的位置
    int n=strlen(s), m=strlen(p); //n为s的长度, m为p的长度
    int i = 0; //通过i扫描s
    while(i<=n-m){ //看p是否在s的第i个位置, 从s[i]开始与p逐字符比对
        int j = 0; //通过j扫描p
        while(j<m && s[i]==p[j]) {
            i++; j++;
        }
        if(j == m) return i - m; // 匹配成功
        i = i - j + 1; //本趟匹配失败, 指针i回退到原位置+1
    }
}
```

.....未完待续

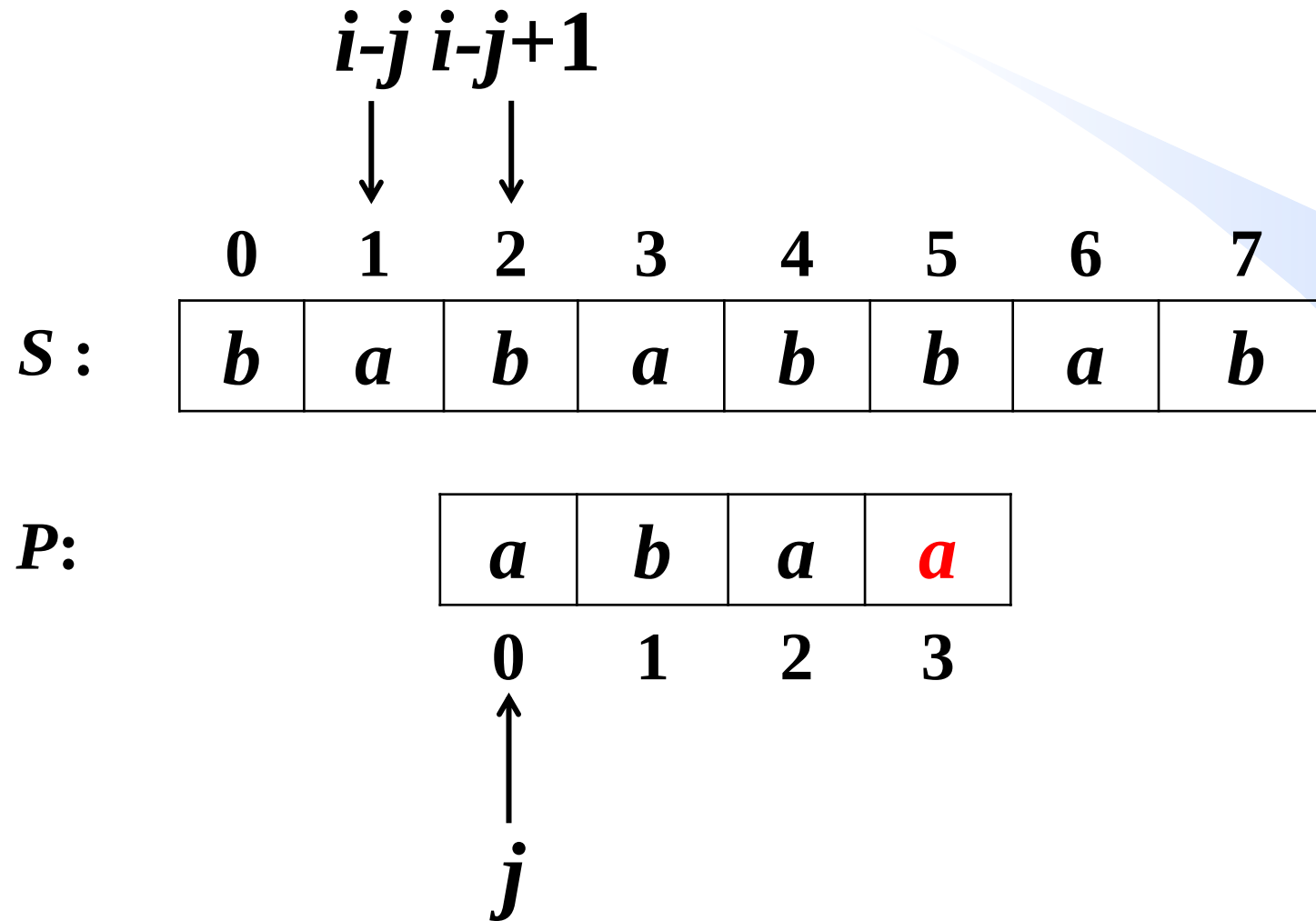
本趟匹配失败的情况



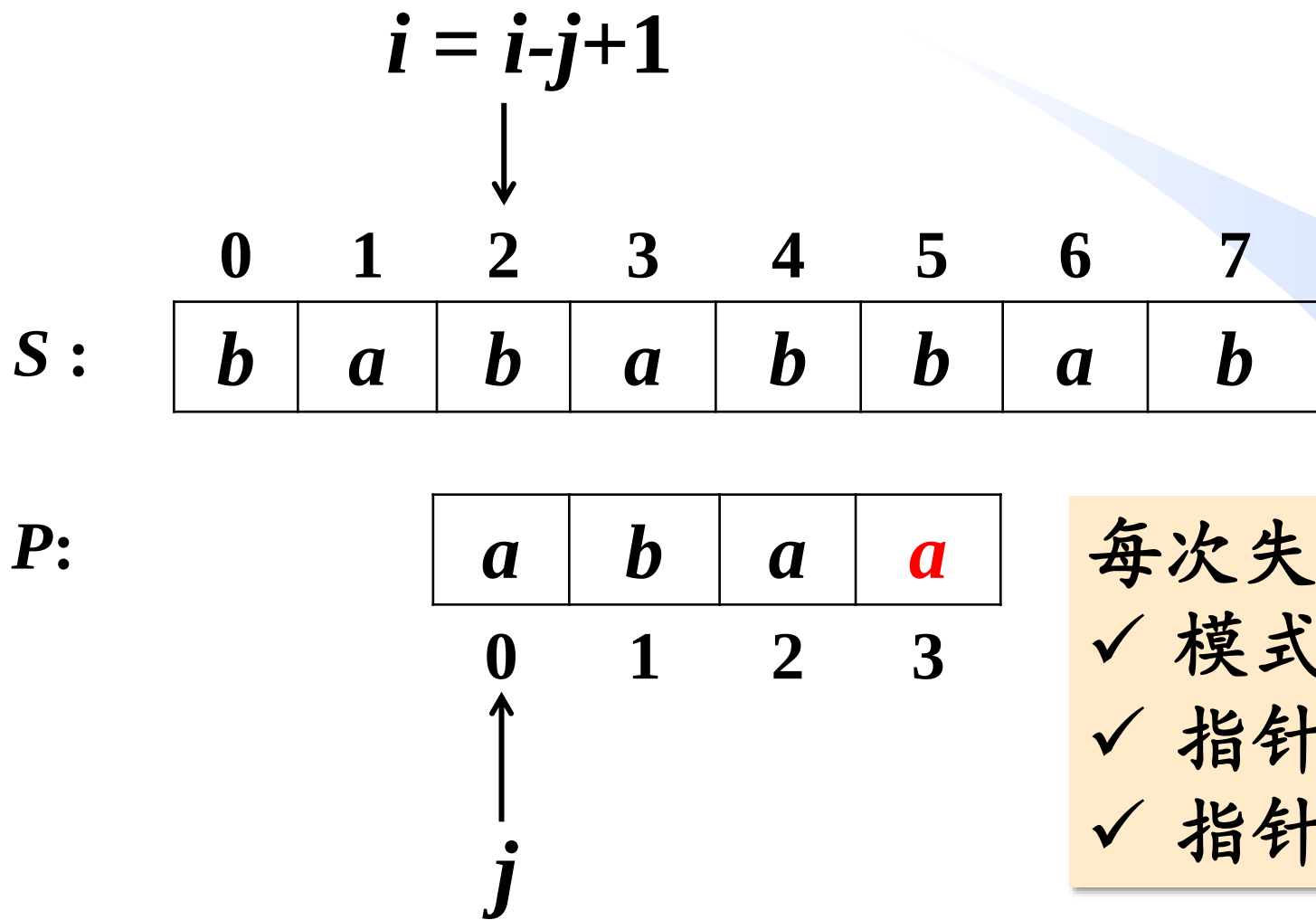
本趟匹配失败的情况



本趟匹配失败的情况



本趟匹配失败的情况



每次失配后：

- ✓ 模式串右移1位
- ✓ 指针 *i* 回退到 $i - j + 1$
- ✓ 指针 *j* 回退到 0

朴素模式匹配算法

```
int StringMatching(char *s, char *p){ //返回p在s中的位置
    int n=strlen(s), m=strlen(p); //n为s的长度, m为p的长度
    int i = 0; //通过i扫描s
    while(i<=n-m){ //看p是否在s的第i个位置, 从s[i]开始与p逐字符比对
        int j = 0; //通过j扫描p
        while(j<m && s[i+j]==p[j]) {
            i++; j++;
        }
        if(j == m) return i - m; // 匹配成功
        i = i - j + 1; //本趟匹配失败, 指针i回退到原位置+1
    }
    return -1; // p不在s中
}
```

关键运算：字符比较
最坏时间复杂度
 $O(n \cdot m)$

朴素模式匹配算法——平均情况时间复杂度

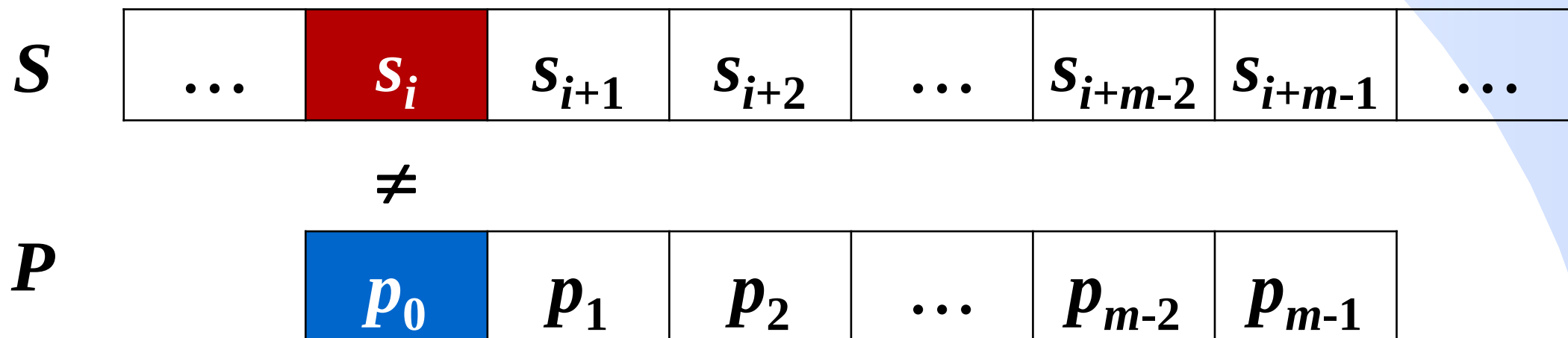
假定目标串 S 长度为 n ，模式串 P 长度为 m ，证明朴素模式匹配算法在平均情况下的字符比较次数不超过 $2n$. 【东京大学考研题】

- 考察第 i 趟匹配：看 P 是否在 S 的第 i 个位置。
- 假定字符集包含 k 种字符，则 P_j 等于 S 中某个字符 S_i 的概率为 $\frac{1}{k}$ ， P_j 不等于 S_i 的概率为 $1 - \frac{1}{k}$ 。

S	...	S_i	S_{i+1}	S_{i+2}	...	S_{i+m-2}	S_{i+m-1}	...
P		p_0	p_1	p_2	...	p_{m-2}	p_{m-1}	

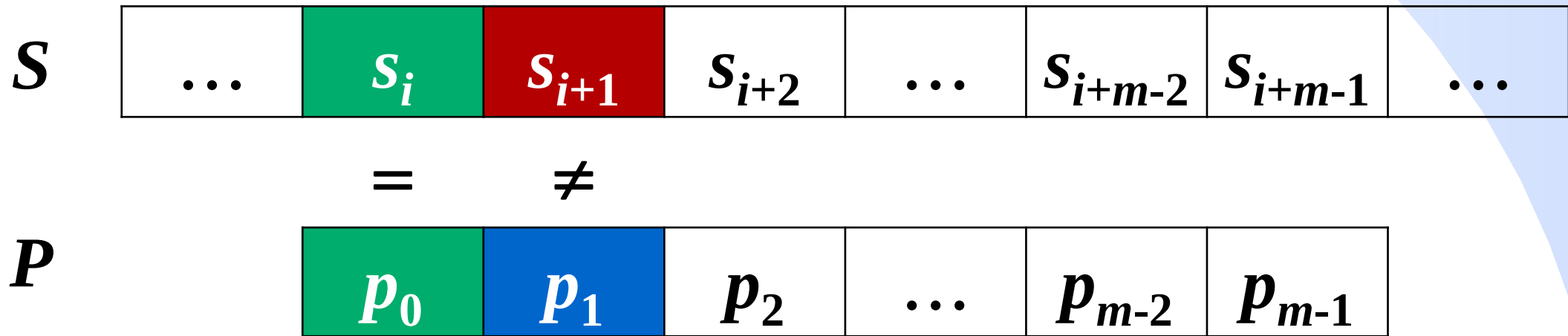
朴素模式匹配算法——平均情况时间复杂度

	字符比较次数	发生概率
第1种输入	1	$1 - \frac{1}{k}$



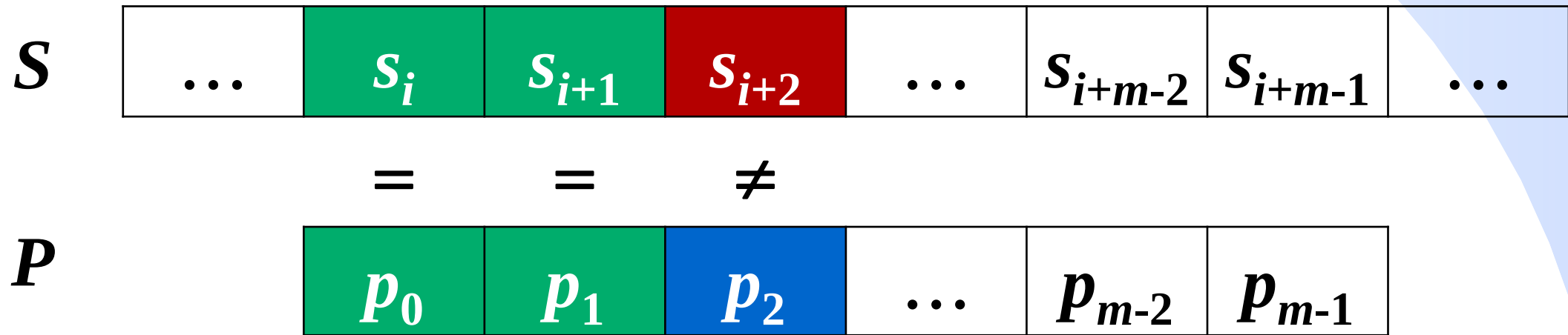
朴素模式匹配算法——平均情况时间复杂度

	字符比较次数	发生概率
第2种输入	2	$\frac{1}{k} \left(1 - \frac{1}{k} \right)$



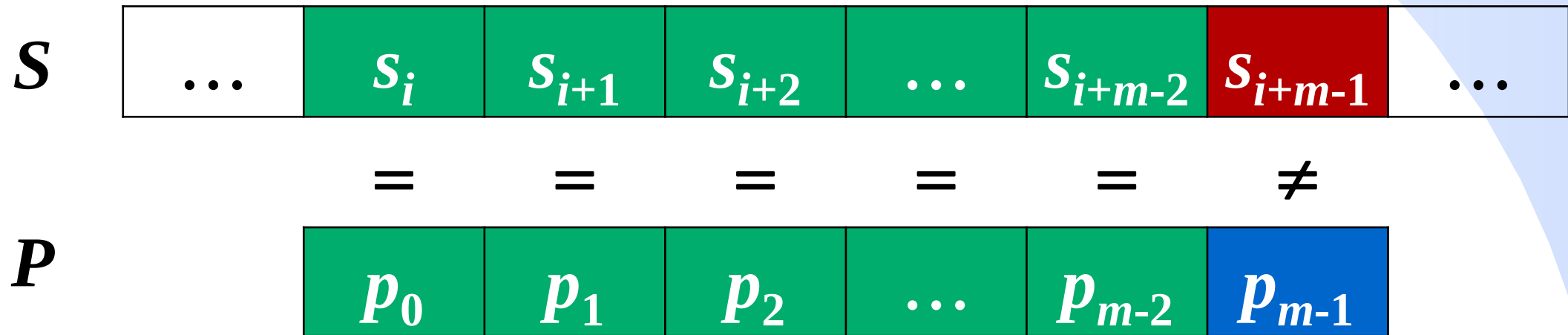
朴素模式匹配算法——平均情况时间复杂度

	字符比较次数	发生概率
第3种输入	3	$\frac{1}{k^2} \left(1 - \frac{1}{k} \right)$



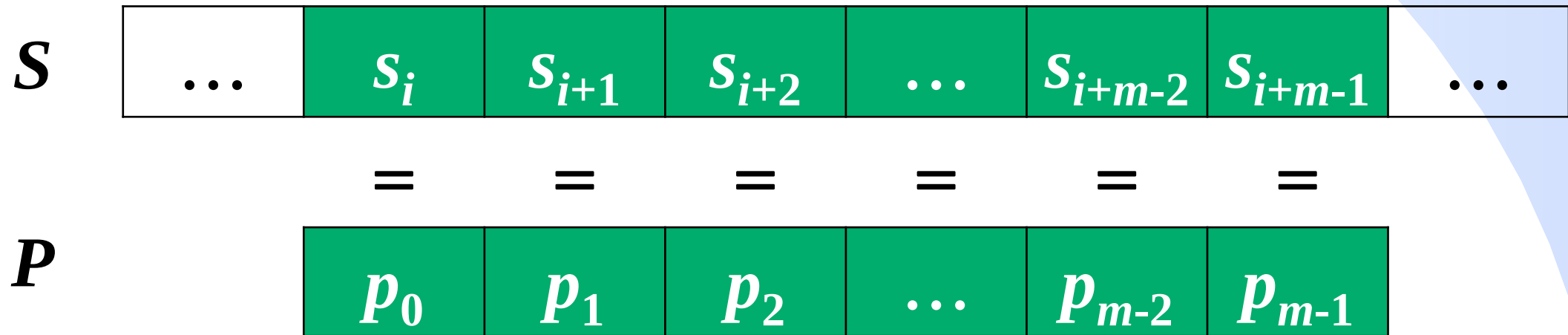
朴素模式匹配算法——平均情况时间复杂度

字符比较次数		发生概率
第 m 种输入	m	$\frac{1}{k^{m-1}} \left(1 - \frac{1}{k} \right)$



朴素模式匹配算法——平均情况时间复杂度

	字符比较次数	发生概率
第 $m+1$ 种输入	m	$\frac{1}{k^m}$



朴素模式匹配算法——平均情况时间复杂度

第*i*趟匹配：看*P*是否在*S*的第*i*个位置。假定字符集包含*k*种字符

	字符比较次数	发生概率
第1种输入	1	$1 - \frac{1}{k}$
第2种输入	2	$\frac{1}{k} \left(1 - \frac{1}{k}\right)$
...	...	...
第 <i>m</i> 种输入	<i>m</i>	$\frac{1}{k^{m-1}} \left(1 - \frac{1}{k}\right)$
第 <i>m</i> +1种输入	<i>m</i>	$\frac{1}{k^m}$

朴素模式匹配算法——平均情况时间复杂度

$$T_i = \left(1 - \frac{1}{k}\right) + \frac{2}{k} \left(1 - \frac{1}{k}\right) + \frac{3}{k^2} \left(1 - \frac{1}{k}\right) + \cdots + \frac{m}{k^{m-1}} \left(1 - \frac{1}{k}\right) + \frac{m}{k^m}$$

$$T_i = \left(1 - \frac{1}{k}\right) \left[1 + \frac{2}{k} + \frac{3}{k^2} + \cdots + \frac{m}{k^{m-1}}\right] + \frac{m}{k^m}$$

$$\text{令 } S = \left(1 + \frac{2}{k} + \frac{3}{k^2} + \cdots + \frac{m}{k^{m-1}}\right) \text{ ①}$$

$$\text{则 } \frac{S}{k} = \left(\frac{1}{k} + \frac{2}{k^2} + \frac{3}{k^3} + \cdots + \frac{m}{k^m}\right) \text{ ②}$$

$$\text{令 ①} - \text{② 有 } \left(1 - \frac{1}{k}\right) S = \left(1 + \frac{1}{k} + \frac{1}{k^2} + \cdots + \frac{1}{k^{m-1}} - \frac{m}{k^m}\right)$$

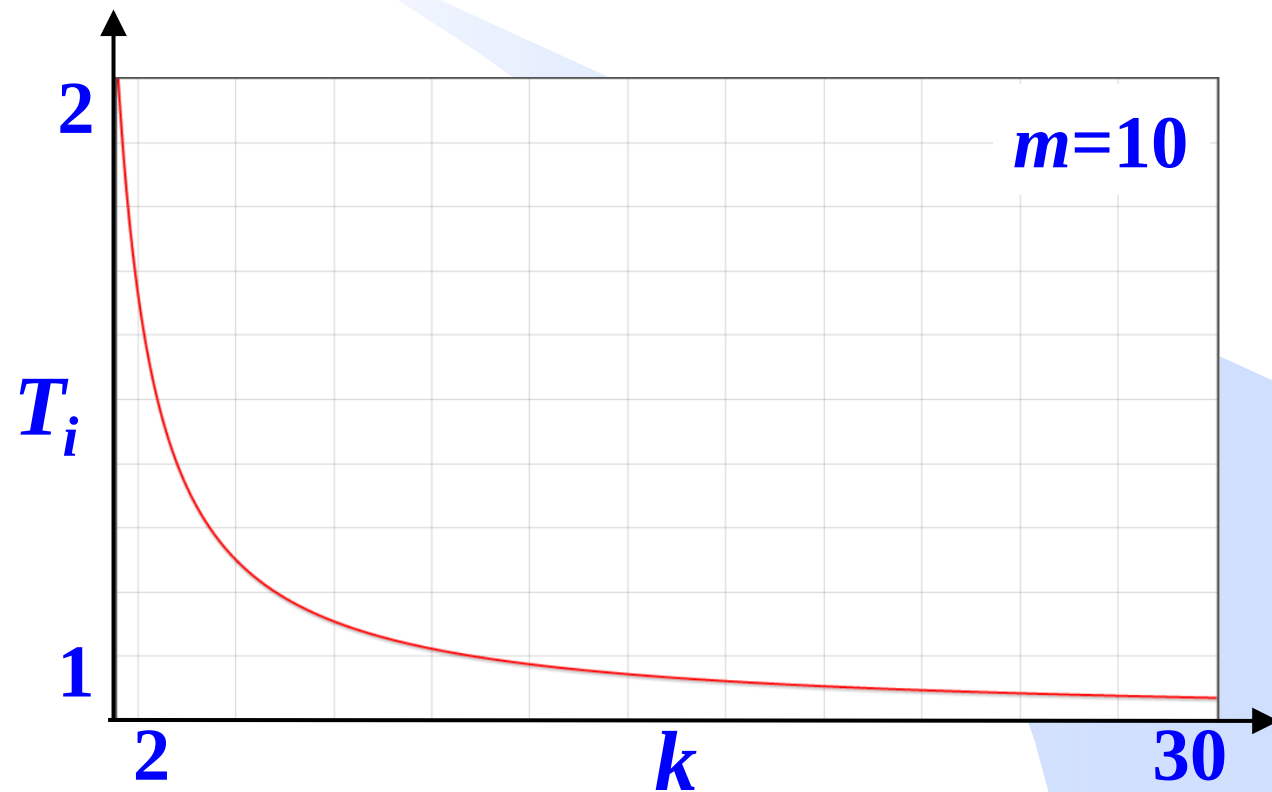
朴素模式匹配算法——平均情况时间复杂度

- 第 i 趟匹配：看 P 是否在 S 的第 i 个位置
- 由于字符集包含 k 种字符，故 $k \geq 2$

$$T_i = 1 + \frac{1}{k} + \frac{1}{k^2} + \cdots + \frac{1}{k^{m-1}}$$

$$= \frac{1 - \left(\frac{1}{k}\right)^m}{1 - \frac{1}{k}} \leq 2$$

$$\therefore T \leq nT_i \leq 2n$$

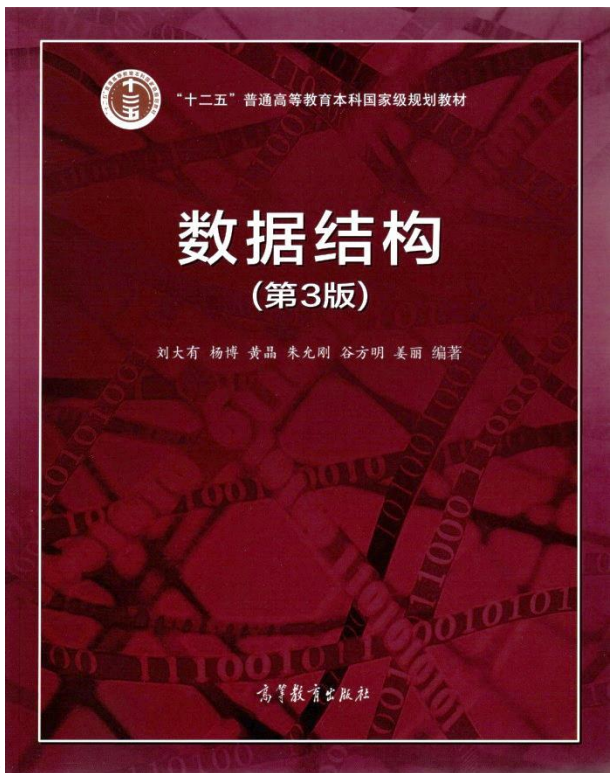


字符集包含的字符种类越多，平均时间复杂度越低

朴素模式匹配算法——总结

- 思路简单、实现简单。
- 缺点：最坏情况下时间复杂度高。
- 字符集越大，单次比对成功的概率越小，时间越接近线性。
- 字符集越小，单次比对成功的概率越高，越接近最坏情况，算法时间性能越差。

S	...	s_i	s_{i+1}	s_{i+2}	...	s_{i+m-2}	s_{i+m-1}	...
P								
	p_0	p_1	p_2	...	p_{m-2}	p_{m-1}		



字符串匹配

- 模式匹配基本概念
- 朴素模式匹配
- **KMP算法**
- KMP算法的改进

数据之法
结构之美
算法之道

快速模式匹配KMP算法



Donald Knuth

图灵奖获得者

斯坦福大学教授

美国科学院院士

美国工程院院士

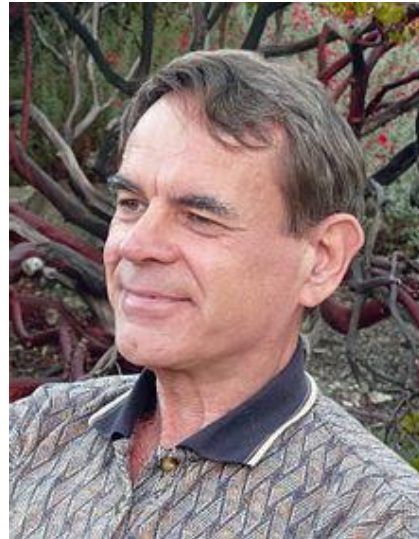
TAOCP, TEX



James Morris

卡内基梅隆大学

教授



Vaughan Pratt

斯坦福大学

教授

SIAM J. COMPUT.
Vol. 6, No. 2, June 1977

FAST PATTERN MATCHING IN STRINGS*

DONALD E. KNUTH[†], JAMES H. MORRIS, JR.[‡] AND VAUGHAN R. PRATT[¶]

Abstract. An algorithm is presented which finds all occurrences of one given string within another, in running time proportional to the sum of the lengths of the strings. The constant of proportionality is low enough to make this algorithm of practical use, and the procedure can also be extended to deal with some more general pattern-matching problems. A theoretical application of the algorithm shows that the set of concatenations of even palindromes, i.e., the language $\{\alpha\alpha^R\}^*$, can be recognized in linear time. Other algorithms which run even faster on the average are also considered.

Key words. pattern, string, text-editing, pattern-matching, trie memory, searching, period of a string, palindrome, optimum algorithm, Fibonacci string, regular expression

Text-editing programs are often required to search through a string of characters looking for instances of a given “pattern” string; we wish to find all positions, or perhaps only the leftmost position, in which the pattern occurs as a contiguous substring of the text. For example, *catenary* contains the pattern *ten*, but we do not regard *canary* as a substring.

The obvious way to search for a matching pattern is to try searching at every starting position of the text, abandoning the search as soon as an incorrect character is found. But this approach can be very inefficient, for example when we are looking for an occurrence of *aaaaaaaaab* in *aaaaaaaaaaaaaaaaab*. When the pattern is a^nb and the text is $a^{2n}b$, we will find ourselves making $(n+1)^2$ comparisons of characters. Furthermore, the traditional approach involves “backing up” the input text as we go through it, and this can add annoying complications when we consider the buffering operations that are frequently involved.

In this paper we describe a pattern-matching algorithm which finds all occurrences of a pattern of length m within a text of length n in $O(m+n)$ units of time, without “backing up” the input text. The algorithm needs only $O(m)$ locations of internal memory if the text is read from an external file, and only

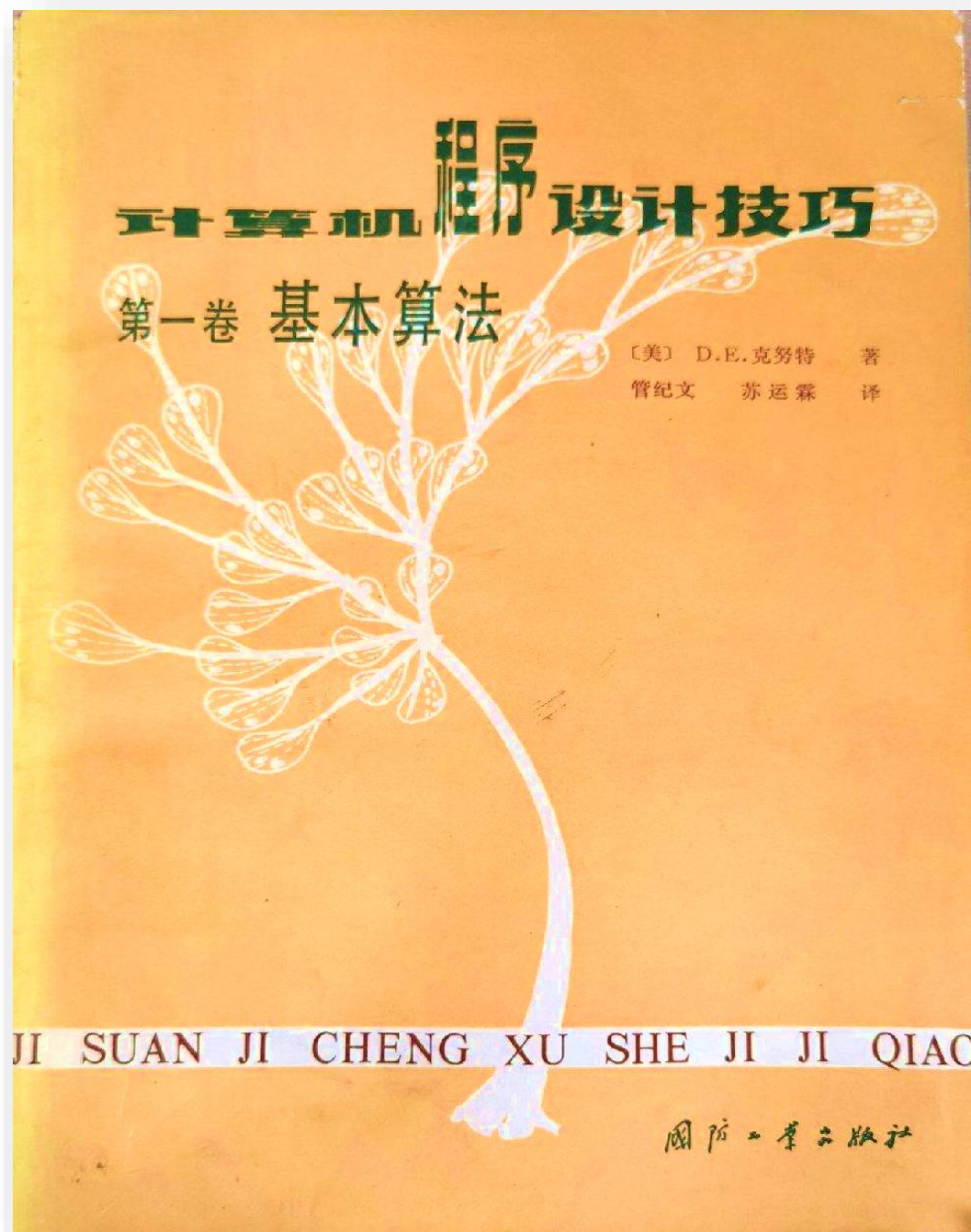
THE CLASSIC WORK
NEWLY UPDATED AND REVISED

The Art of Computer Programming

VOLUME 1

Fundamental Algorithms
Third Edition

DONALD E. KNUTH



THE ART OF COMPUTER PROGRAMMING
Volume 1/Fundamental Algorithms
D.E. Knuth
1973 BY ADDISON-WESLEY PUBLISHING
COMPANY, INC.

计算机程序设计技巧
(第一卷 基本算法)

〔美〕 D.E. 克努特 著
管纪文 苏运霖 译

国防工业出版社 出版

新华书店北京发行所发行 各地新华书店经售
国防工业出版社印刷厂印装

787×1092¹/₁₆ 印张36³/₄ 849千字

1980年3月第一版 1980年3月第一次印刷 印数: 00,001—40,200册

统一书号: 15034·1873 定价: 3.75元



管纪文，原吉林大学计算机科学系主任，中国人工智能学会副理事长，国务院学位委员会学科评议组成员，英国女王大学教授。

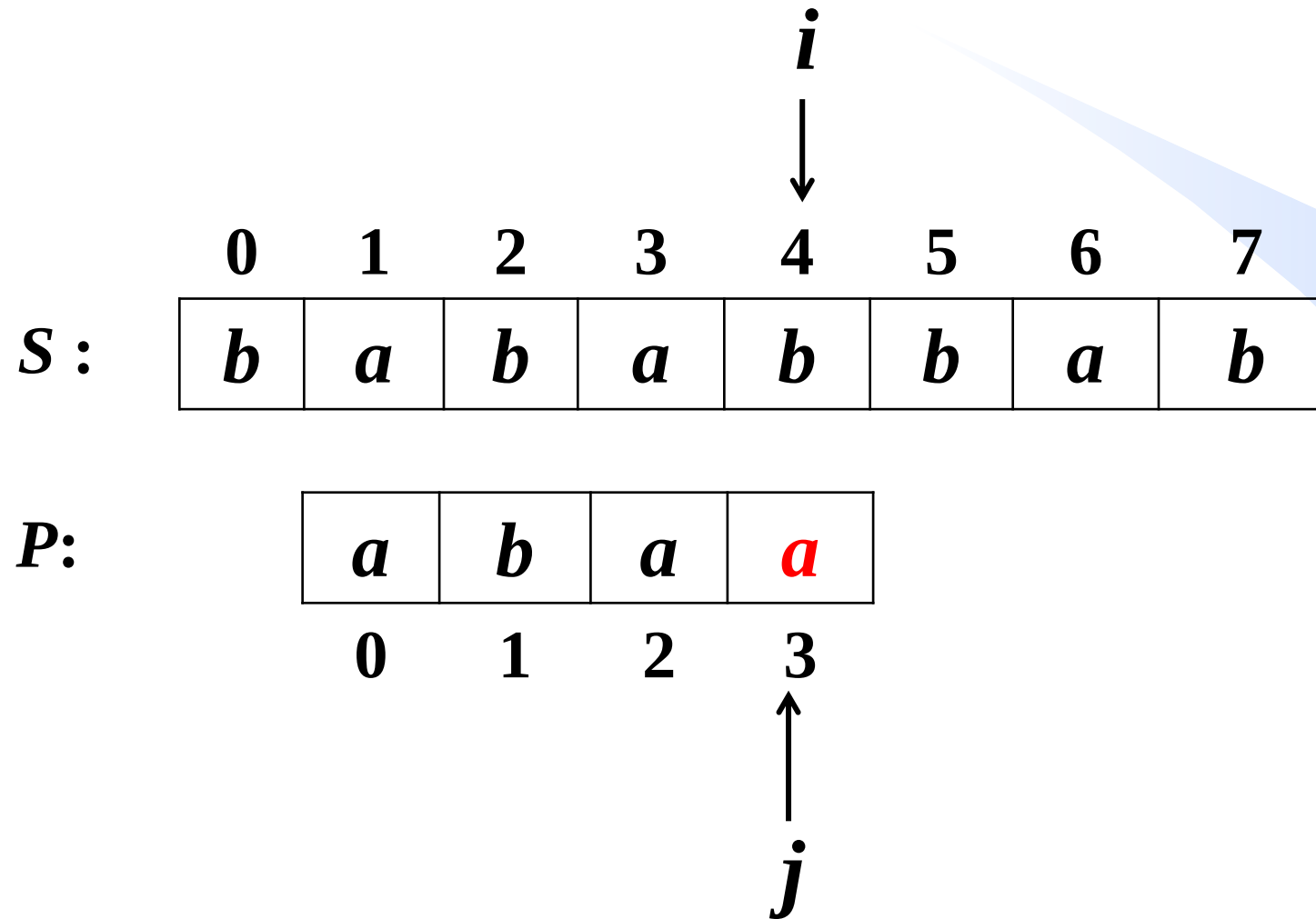


苏运霖，原吉林大学计算机科学系教师，吉林省计算机学会秘书长，后调入暨南大学，是暨南大学计算机学科的主要创建者之一。

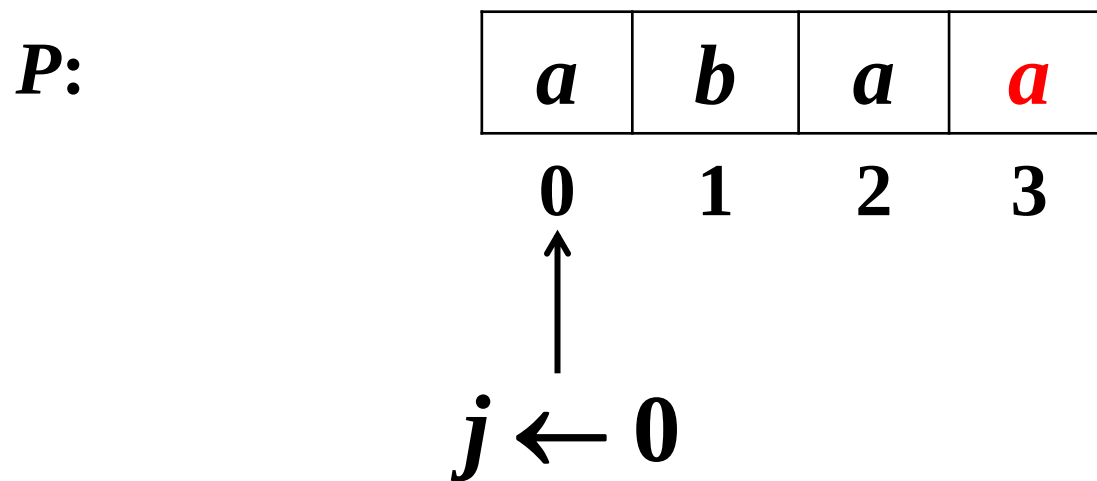
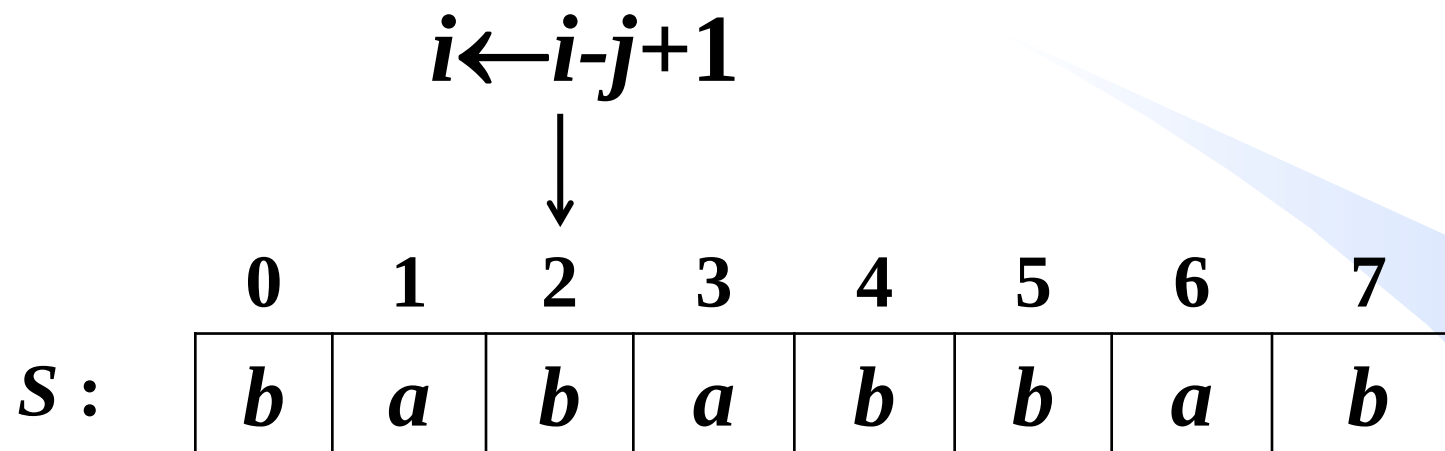


陆汝铃，中国科学院院士，我国知识工程研究的开拓者和先驱之一，曾任中科院数学所副所长，获吴文俊人工智能最高成就奖、中国计算机学会终身成就奖。

本趟匹配失败的情况



本趟匹配失败的情况



每次失配后：
 模式串右移1位
 指针 *i* 回退到 $i-j+1$
 指针 *j* 回退到 0

动机

朴素模式匹配存在的问题
(每次匹配失败后)

可能的优化途径

模式串 P 仅右移1位



模式串 P 能否多移几位?

目标串 S 的匹配指针 i 回退



指针 i 可否不回退?

模式串 P 的匹配指针 j 回退至0



指针 j 可否不回退至0?

利用模式串中子串的重复性，加速匹配过程

KMP算法

A

$S: a \ b \ c \ x \ a \ b \ c \mid x \ a \ b \ c \ y \dots$
 $P: \boxed{a \ b \ c} \ x \ \boxed{a \ b \ c} \mid y$

✓相等前后缀 abc
✓下次比对: P 右移4位

$S: a \ b \ c \ x \ a \ b \ c \mid x \ a \ b \ c \ y \dots$
 $P: \quad \quad \boxed{a \ b \ c} \mid x \ a \ b \ c \ y$

KMP算法

$S: a \ b \ c \ a \ b \ c \ a \ b \ c \ a \ b \ c \ x$
 $P: a \ b \ c \ a \ b \ c \ a \ b \ c \ x$

方案1: 选取相等前后缀 abc , P 右移6位进行下次比对

$S: a \ b \ c \ a \ b \ c \ a \ b \ c \ a \ b \ c \ x$
 $P: \quad \quad \quad a \ b \ c \ a \ b \ c \ a \dots$

漏掉了可能的成功匹配位置!



KMP算法

$S: a \ b \ c \ a \ b \ c \ a \ b \ c \ a \ b \ c \ x$
 $P: a \ b \ c \ a \ b \ c \ a \ b \ c \ x$

方案2：选取更长的相等前后缀 $abcabc$ ， P 右移3位进行下次比对

$S: a \ b \ c \ a \ b \ c \ a \ b \ c \ a \ b \ c \ x$
 $P: \quad \quad \quad a \ b \ c \ a \ b \ c \ a \ b \ c \ x$

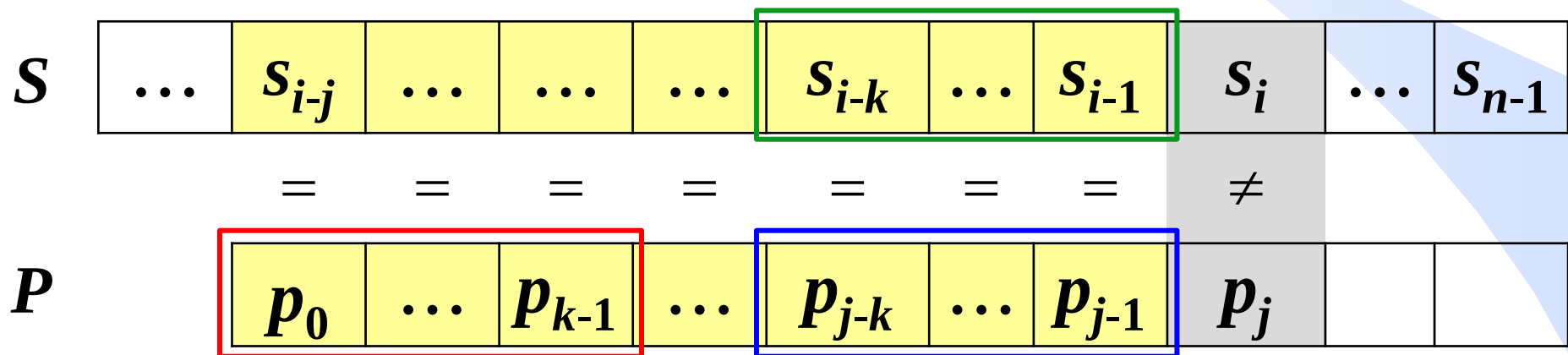
应该保守一些，选择右移位数较少的方案，避免漏掉可能的成功匹配



找 P 失配位置前的子串的最长相等前后缀

KMP算法——基本原理

- 设目标 $S = s_0 s_1 \dots s_{n-1}$, 模式串 $P = p_0 p_1 \dots p_{m-1}$
- 假设当前正进行某次匹配, 有 $s_{i-j} \dots s_{i-1} = p_0 \dots p_{j-1}$, 但 $s_i \neq p_j$, 即在 p_j 的位置失配。



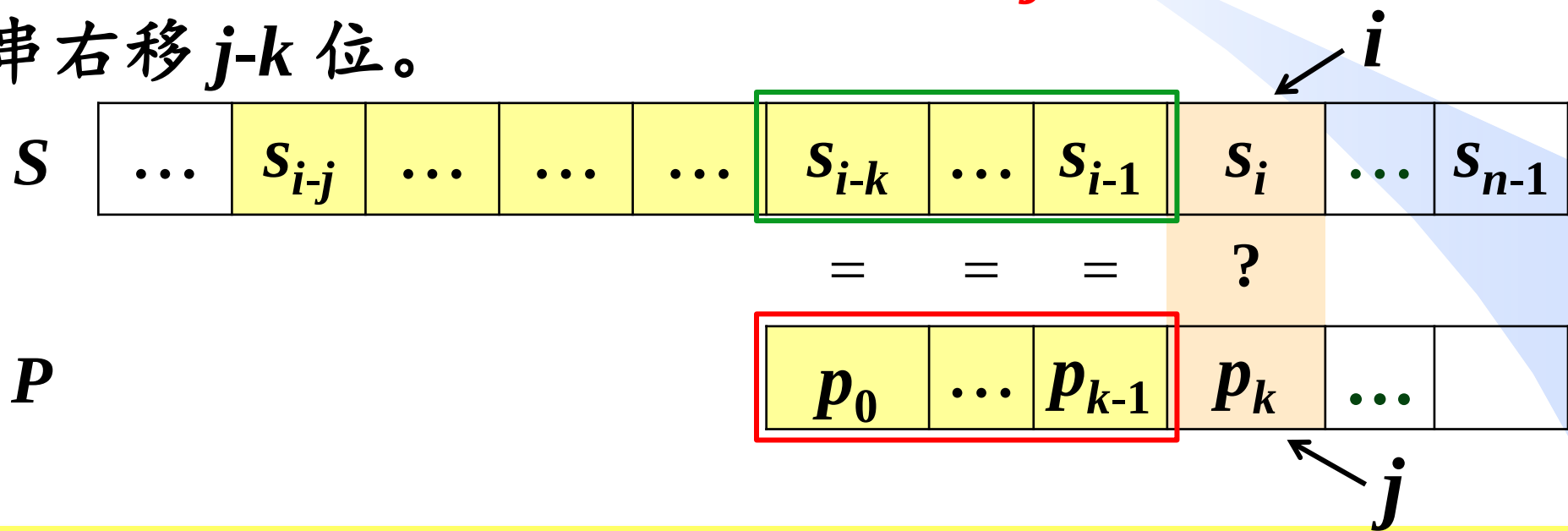
在失配位置前对应的子串 $p_0 \dots p_{j-1}$ 中, 找最长相等的前后缀, 设其长度为 k , 则有:

$$s_{i-k} \dots s_{i-1} = p_{j-k} \dots p_{j-1} = p_0 \dots p_{k-1}$$

KMP算法——基本原理

下次匹配时：

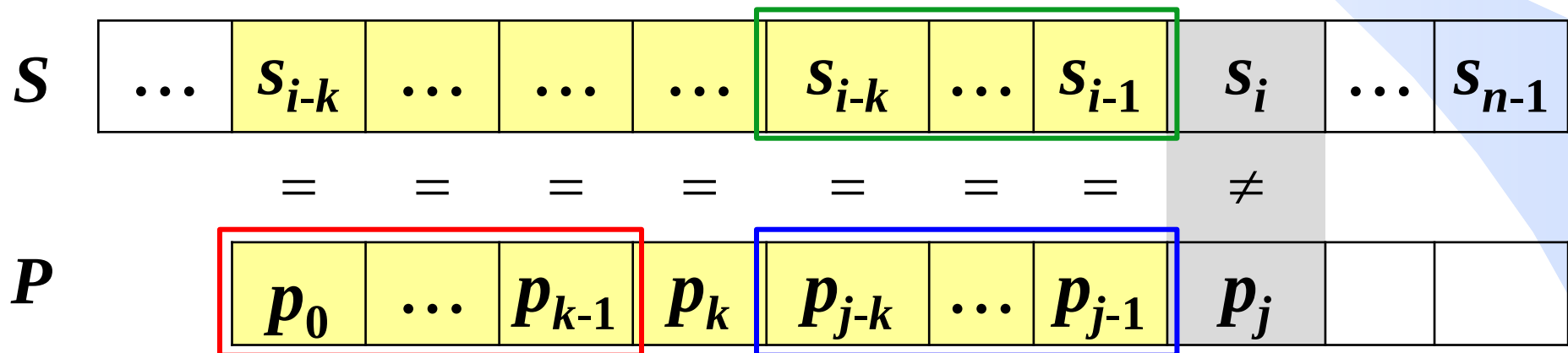
- 右移 P 使 $p_0 \dots p_{k-1}$ 与 $s_{i-k} \dots s_{i-1}$ 对齐，使 s_i 和 p_k 继续比对。
- 目标串匹配指针 i 不动，模式串匹配 j 指针只需回退到 k ，即模式串右移 $j-k$ 位。



k 标识了模式串 P 在位置 j 失配后，下次匹配的位置
 k 是 $p_0 \dots p_{j-1}$ 的最长相等前后缀的长度

在 p_j 处匹配失败后

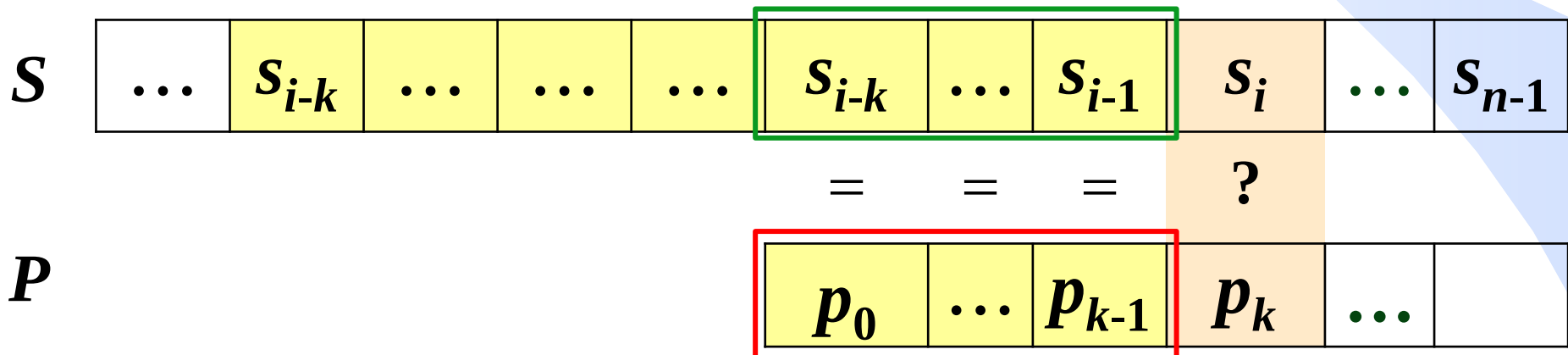
P 的下一个匹配位置 k (P 中的哪个字符和 S 中的失配字符重新匹配) 可看做关于 j 的函数, 即 $k = next(j) = p_0 \dots p_{j-1}$ 的最长相等前后缀的长度。



在 p_j 处匹配失败后

P 的下一个匹配位置 k (P 中的哪个字符和 S 中的失配字符重新匹配) 可看做关于 j 的函数, 即 $k = next(j) = p_0 \dots p_{j-1}$ 的最长相等前后缀的长度。

可以通过预处理, 把所有的 $next(j)$ 算出来



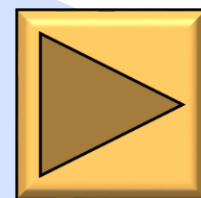
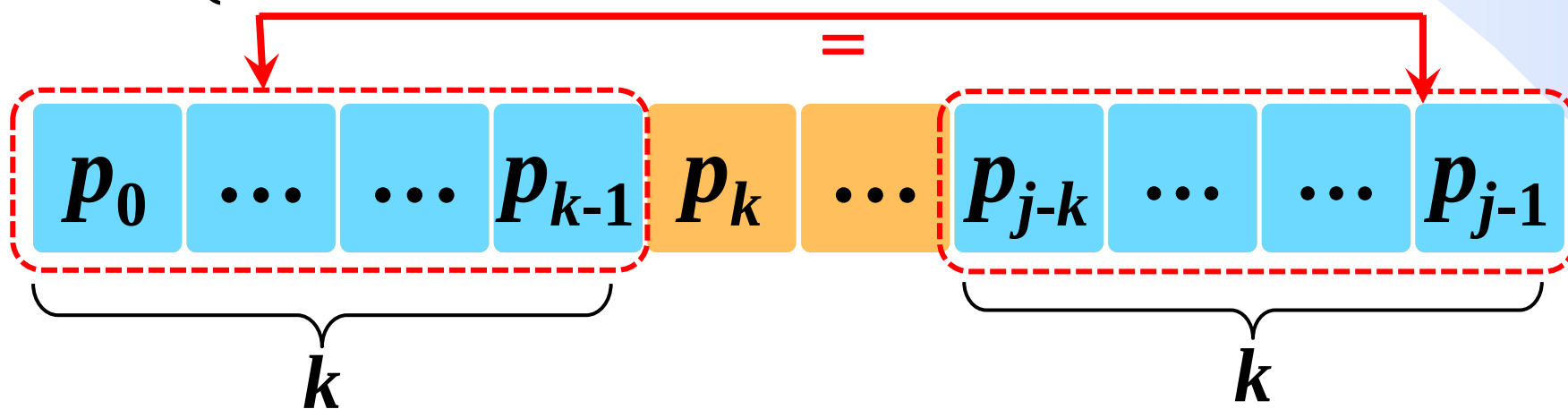
匹配过程中若在 p_j 处失配, 则 P 下次匹配的位置即 $next(j)$

模式串右移
 $j - next(j)$ 位

$next(j)$: p_j 前面的子串的最长相等前后缀的长度

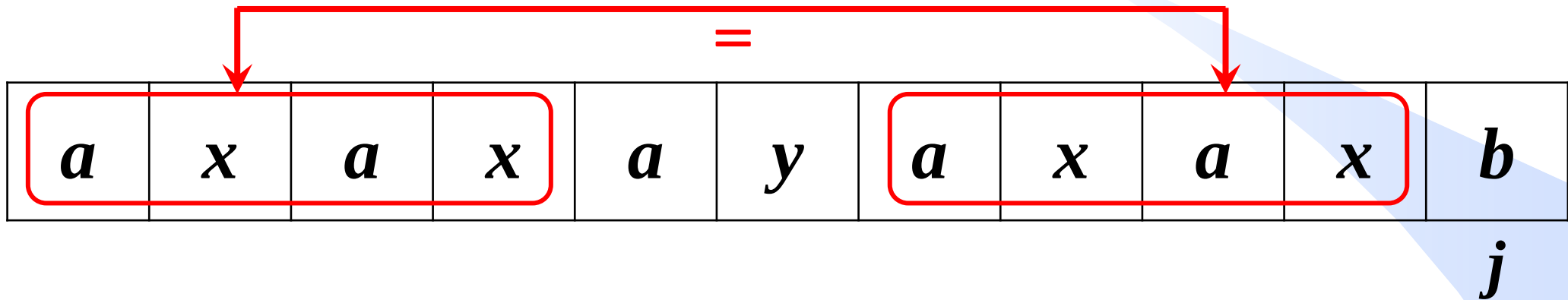
$next$ 函数（失败函数、前缀函数）

$$next(j) = \begin{cases} -1, & j = 0 \\ \max\{k | p_0 \dots p_{k-1} = p_{j-k} \dots p_{j-1} \text{ 且 } 0 < k < j\}, & \text{存在 } k \\ 0, & \text{其他} \end{cases}$$



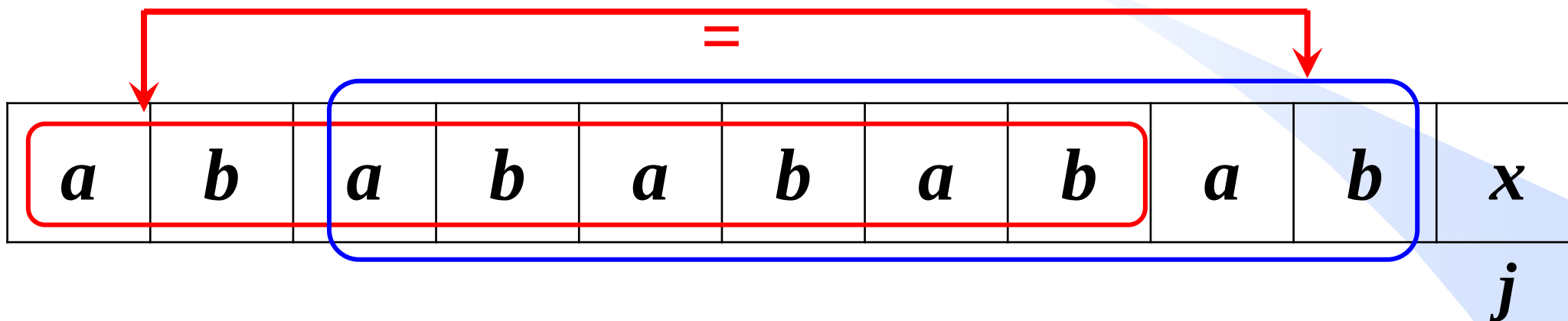
- ✓ $next$ 函数仅与模式串 P 有关，而与目标串 S 无关。
- ✓ 可以预先把 P 中所有位置的 $next(j)$ 算出来，存入数组 $next[j]$ 。
- ✓ 匹配过程中若在 P_j 失配，则直接令 $j \leftarrow next[j]$ 作为 P 下次匹配的位置。

*next*函数——例1



$next(j) = 4$

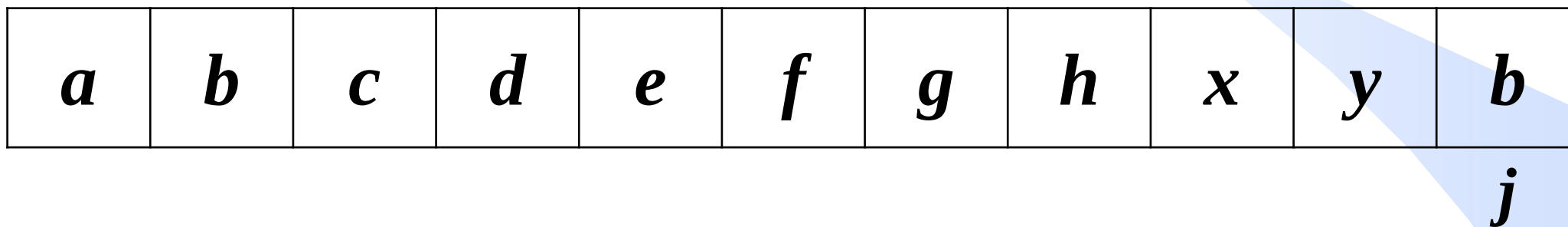
next函数——例2



***next* (*j*) = 8**

*next*函数——例3

A



next (*j*) = 0

练习

模式串为 $abcabcaaa$ ，使用KMP算法进行模式匹配过程中，若模式串在位置 $j=7$ 处与目标串匹配失败，则下一次匹配时，应该是模式串的_____位置与目标串失配位置进行匹配。本题下标从0开始。【吉林大学计软两院2021级期末考试题】

A. 0

B. 2

C. 3

D. 4

<i>a</i>	<i>b</i>	<i>c</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>a</i>	<i>a</i>	<i>a</i>
0	1	2	3	4	5	6	7	8

 j

$$next(j) = next(7) = 4$$

课下思考

模式串为 *abaac*，使用KMP算法进行模式匹配，在主串某处失配后，模式串右移的最长距离为_____。【吉林大学计软两院2022级期末考试题】

A. 0

B. 3

C. 2

D. 4

P

j

next[j]

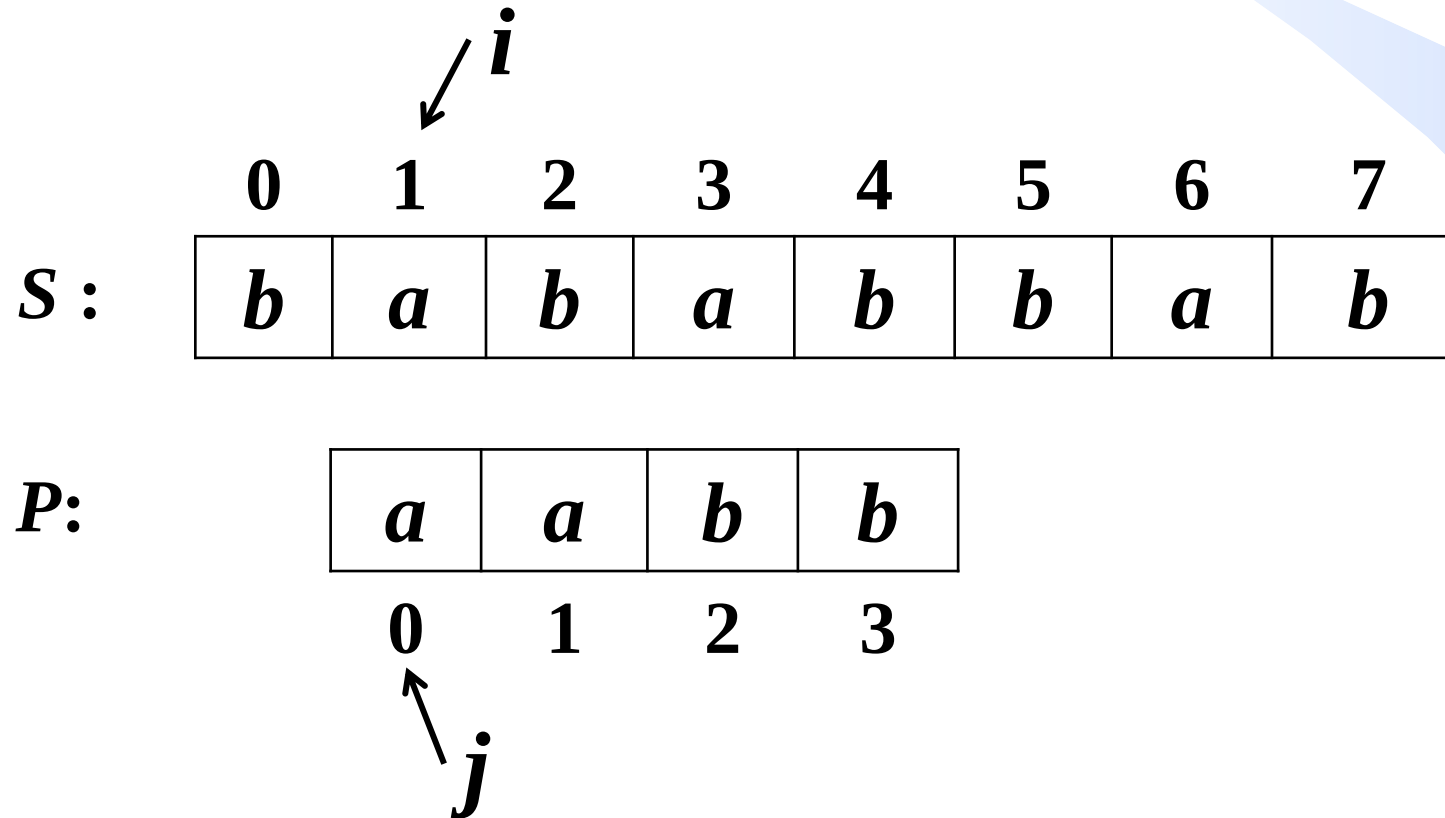
右移距离: $j - next[j]$

<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>c</i>
0	1	2	3	4
-1	0	0	1	1
1	1	2	2	3

KMP算法——基本过程

通过指针*i*扫描*S*，指针*j*扫描*P*

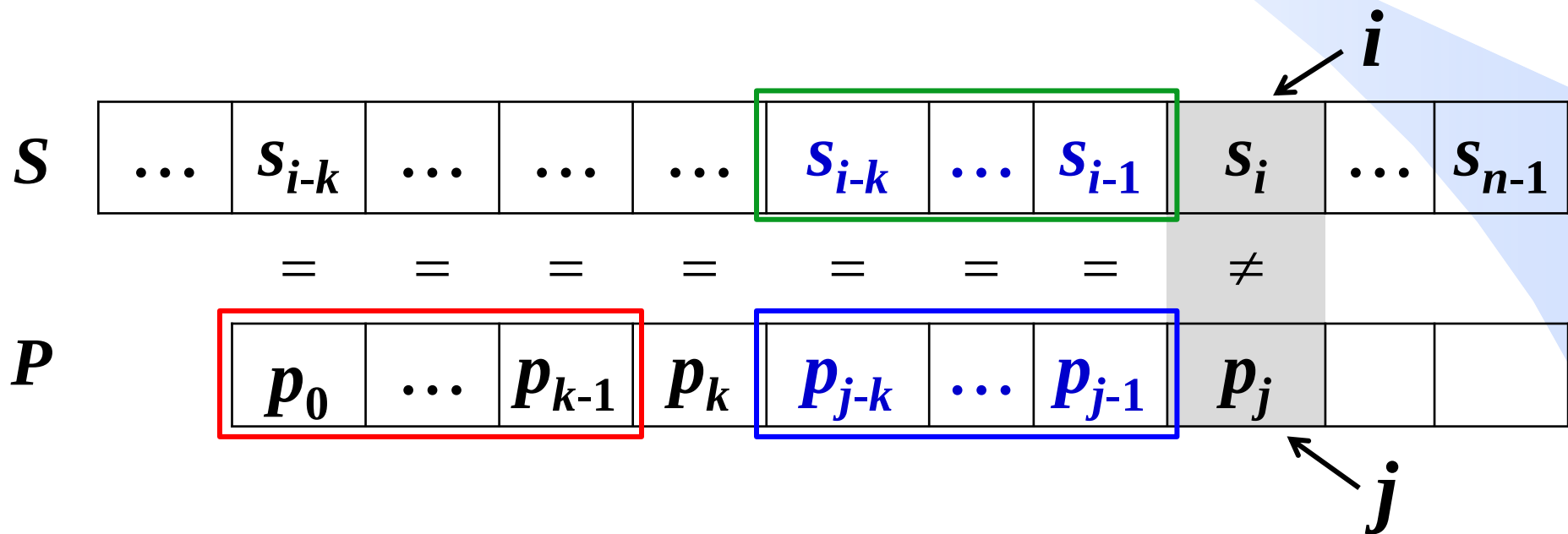
➤ 如果 $s_i = p_j$ ：指针*i*和指针*j* 右移1位继续匹配



KMP算法——基本过程

通过指针*i*扫描*S*，指针*j*扫描*P*

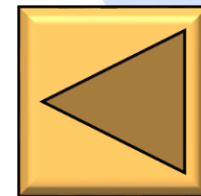
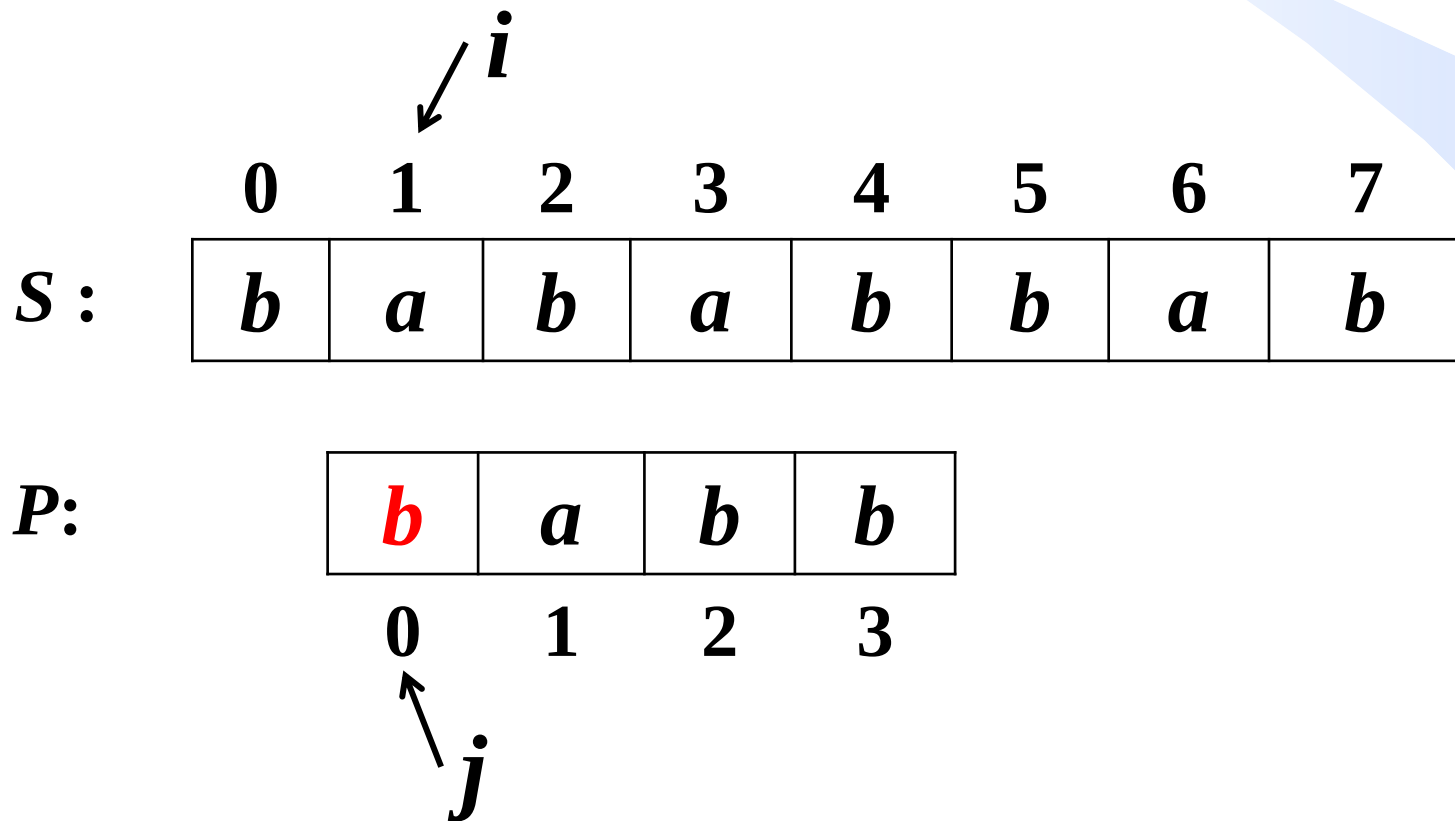
➤ 在 p_j ($j > 0$)处失配后：指针*i*不动，指针 $j \leftarrow \text{next}[j]$ ，继续匹配



KMP算法——基本过程

特例：在 p_0 处失配

➤ 指针 i 右移1位，指针 j 置为0，即模式串右移1位。



KMP算法

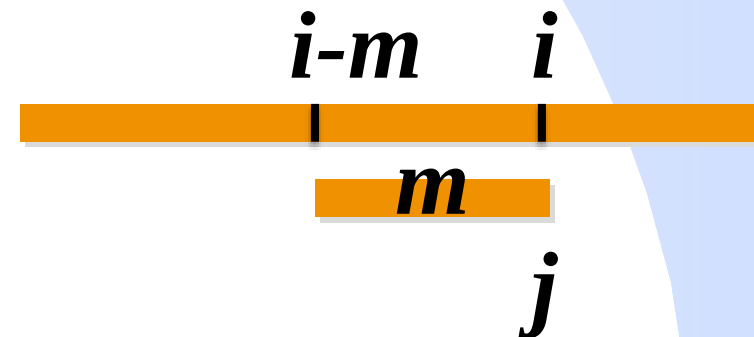
```

const int maxm = 1e4+10;
int KMP(char *S, char *P, int n, int m){
    //目标串S长度为n, 模式串P长度为m, 返回P在S中的位置
    int next[maxm], i=0, j=0;    // i,j分别为S和P的匹配指针
    buildNext(P, next, m);      //计算next函数
    while(j<m && i<n) {
        if(S[i]==P[j]) i++, j++;
        else if(j==0) i++;
        else j=next[j]; //确定P下次匹配位置
    }
    return (j==m)? i-m : -1;
}

```

匹配成功

匹配失败



KMP算法

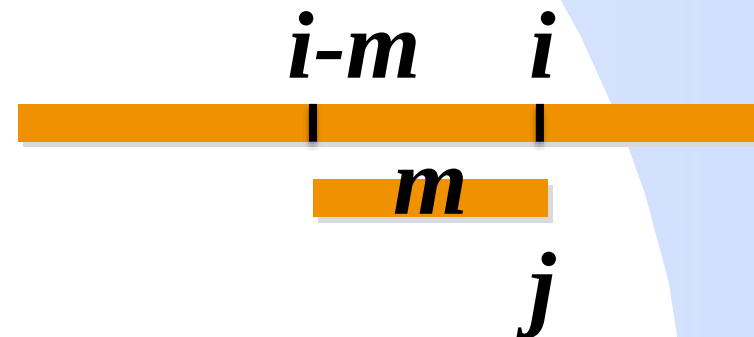
```

const int maxm = 1e4+10;
int KMP(char *S, char *P, int n, int m){
    //目标串S长度为n, 模式串P长度为m, 返回P在S中的位置
    int next[maxm], i=0, j=0;    // i,j分别为S和P的匹配指针
    buildNext(P, next, m);      //计算next函数
    while(j<m && i<n) {
        if(j==-1 || S[i]==P[j]) i++, j++;
        else j=next[j]; //确定P下次匹配位置
    }
    return (j==m)? i-m : -1;
}

```

匹配成功

匹配失败



*next*函数的计算——练习

$j =$	0	1	2	3	4	5	6
$P =$	a	b	c	a	b	c	d
$next(j) =$	-1	0	0	0	1	2	3

编程实现 $next(j)$ 的计算——暴力方法

- 在 $p_0 p_1 \dots p_{j-1}$ 中找**最长相等的前后缀**
- 暴力方案：从长到短考察每个前缀，看能否匹配后缀。
- 例如：模式串 $a b a a b$

长度为4的前缀 $a b a a$	$b a a b$ 长度为4的后缀
长度为3的前缀 $a b a$	$a a b$ 长度为3的后缀
长度为2的前缀 $a b$	$a b$ 长度为2的后缀

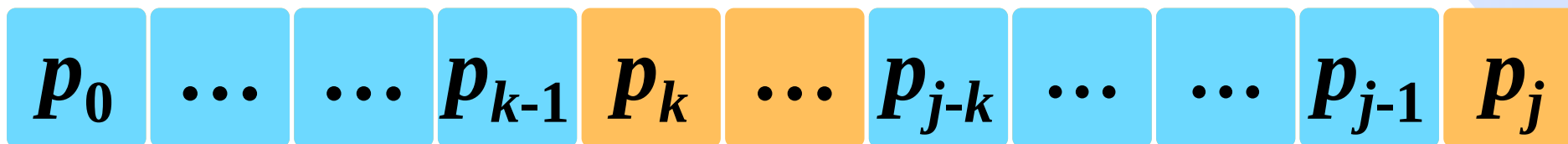
$O(m^2)$

函数 $next(j)$ 的计算——高效方法

➤ 递推计算 $next$ 函数:

✓ 令 $next(0) = -1$;

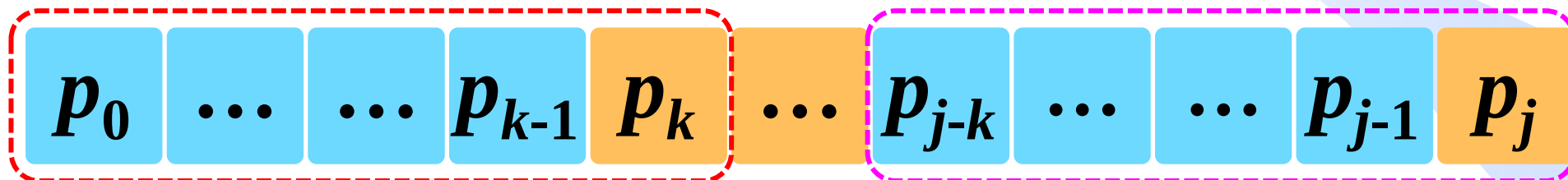
✓ 假定已知 $next(j)=k$, 即 $p_0 \dots p_{j-1}$ 的最长相等前后缀为 $p_0 \dots p_{k-1} = p_{j-k} \dots p_{j-1}$, 其长度为 k ;



✓ 求 $next(j+1)$, 即 $p_0 \dots p_j$ 的最长相等前后缀的长度。

已知 $next(j)=k$, 求 $next(j+1)$

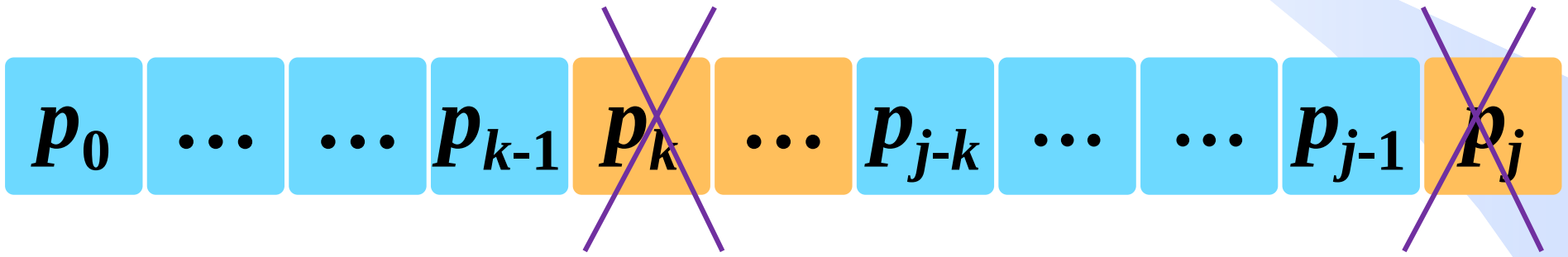
如果 $p_k = p_j$



- ✓ $p_0 \dots p_j$ 的最长相等前后缀为 $p_0 \dots p_k = p_{j-k} \dots p_j$
- ✓ 其长度 $next(j+1) = k+1$

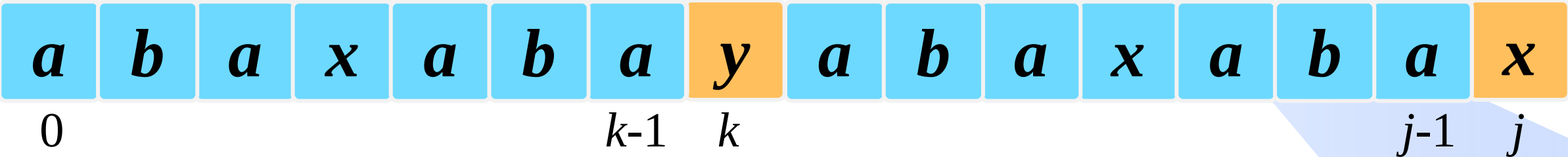
已知 $next(j)=k$, 求 $next(j+1)$

如果 $p_k \neq p_j$



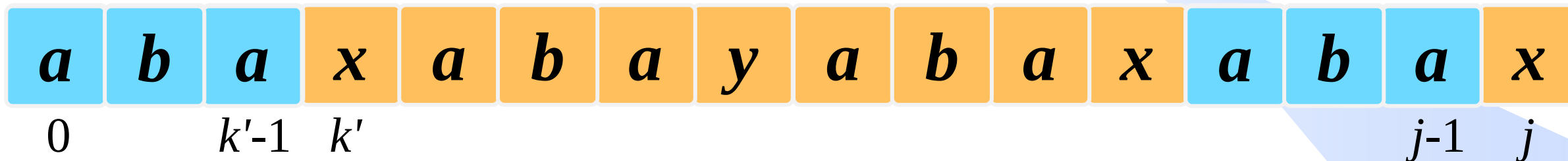
已知 $next(j)=k$, 求 $next(j+1)$

如果 $p_k \neq p_j$



已知 $next(j)=k$, 求 $next(j+1)$

如果 $p_k \neq p_j$

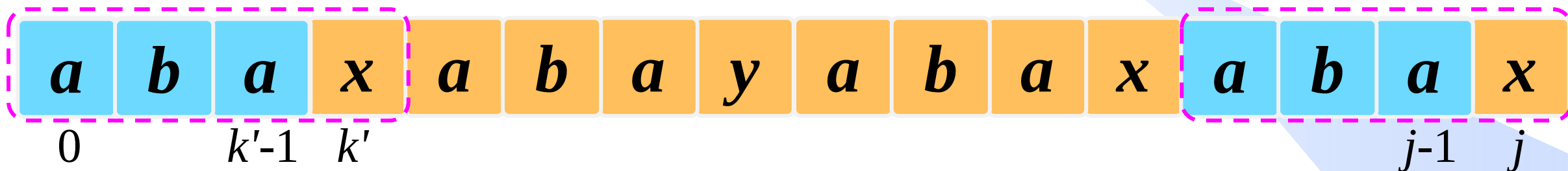


考察 $p_0 \dots p_{j-1}$ 中稍短一点（第2长）的相等前后缀（假定其长度为 k' ），然后比较这两个前后缀的下一个字符 $p_{k'}$ 和 p_j ：

✓ 若 $p_{k'} = p_j$ ：则 $next[j+1] = k' + 1$

已知 $next(j)=k$, 求 $next(j+1)$

如果 $p_k \neq p_j$

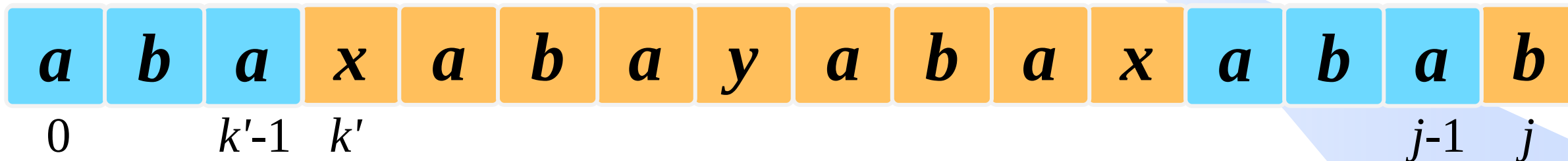


考察 $p_0 \dots p_{j-1}$ 中稍短一点（第2长）的相等前后缀（假定其长度为 k' ），然后比较这两个前后缀的下一个字符 $p_{k'}$ 和 p_j ：

✓ 若 $p_{k'} = p_j$ ：则 $next[j+1] = k' + 1$

已知 $next(j)=k$, 求 $next(j+1)$

如果 $p_k \neq p_j$

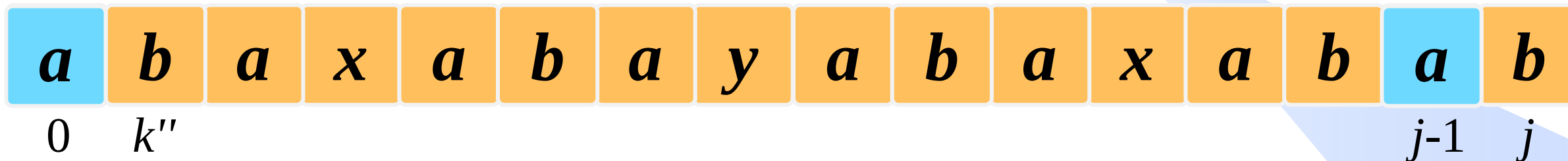


考察 $p_0 \dots p_{j-1}$ 中稍短一点（第2长）的相等前后缀（假定其长度为 k' ），然后比较这两个前后缀的下一个字符 $p_{k'}$ 和 p_j ：

- ✓ 若 $p_{k'} = p_j$ ：则 $next[j+1] = k' + 1$
- ✓ 若 $p_{k'} \neq p_j$ ：见下页

已知 $next(j)=k$, 求 $next(j+1)$

如果 $p_k \neq p_j$

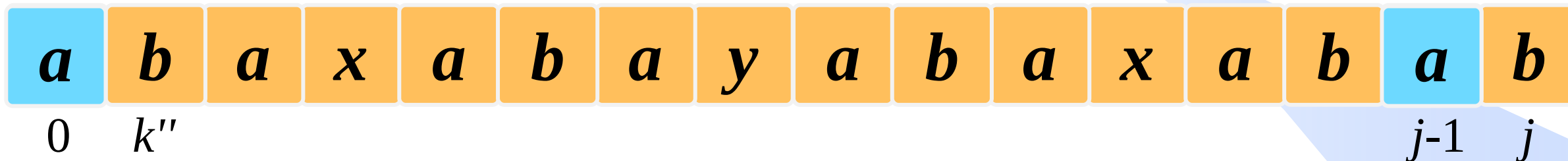


考察 $p_0 \dots p_{j-1}$ 中更短一点（第3长）的相等前后缀（假定其长度为 k'' ），然后比较这两个前后缀的下一个字符 $p_{k''}$ 和 p_j ：

- ✓ 若 $p_{k''} = p_j$ ：则 $next[j+1] = k'' + 1$
- ✓ 若 $p_{k''} \neq p_j$ ：考察 $p_0 \dots p_{j-1}$ 更短一点（第4长）的相等前后缀

已知 $next(j)=k$, 求 $next(j+1)$

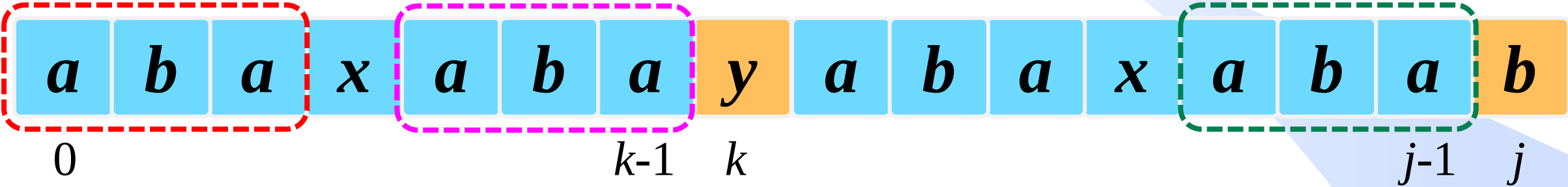
如果 $p_k \neq p_j$



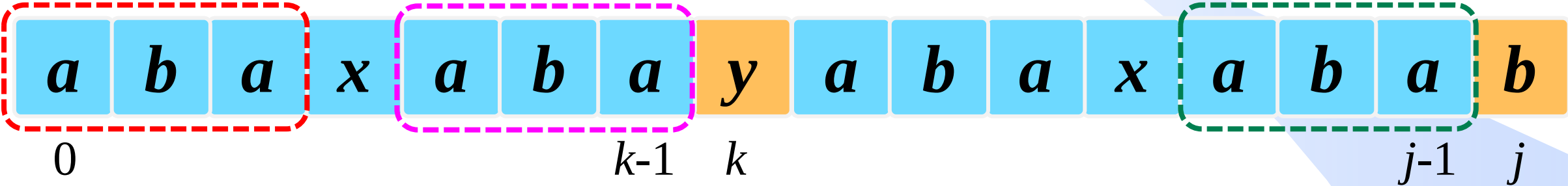
while ($p_k \neq p_j$)

$k = p_0 \dots p_{j-1}$ 中更短一点的相等前后缀的长度;
 $next[j+1] = k+1;$

已知 $next(j)=k$, 如何找 $p_0 \dots p_{j-1}$ 中更短的相等前后缀

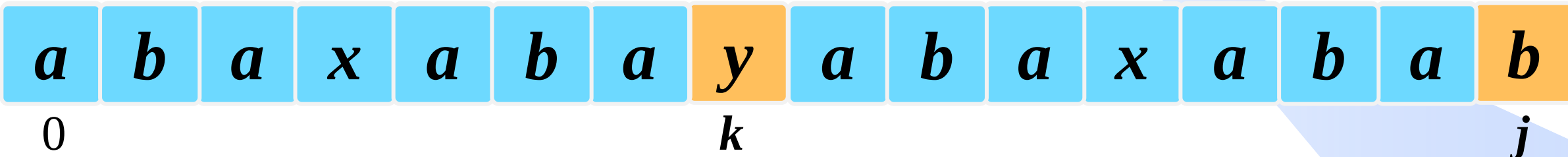


已知 $next(j)=k$, 如何找 $p_0 \dots p_{j-1}$ 中更短的相等前后缀



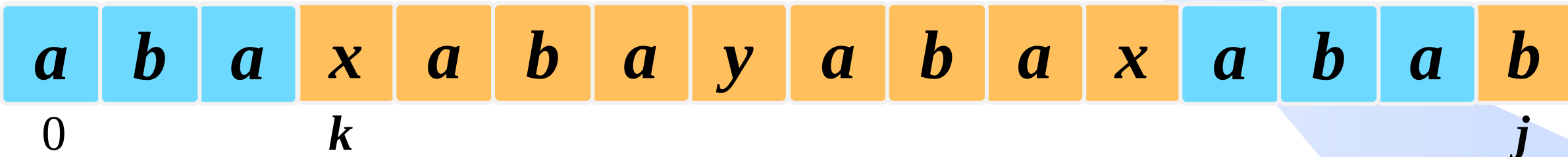
```
while (k >= 0 && p[k] != p[j])  
    k = next[k];  
next[j+1] = k+1;
```

已知 $next(j)=k$, 求 $next(j+1)$



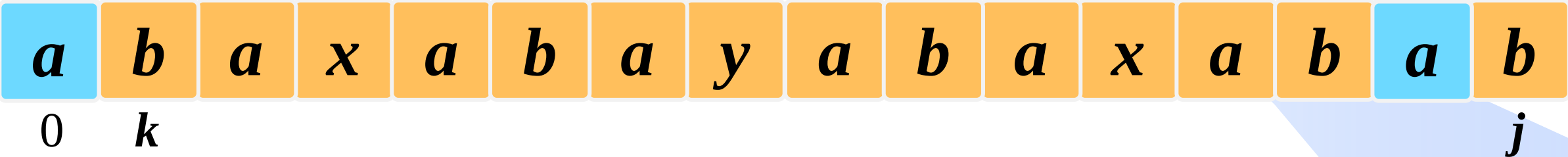
```
while(k >= 0 && p[k] != p[j])  
    k = next[k];  
next[j+1] = k+1;
```

已知 $next(j)=k$, 求 $next(j+1)$



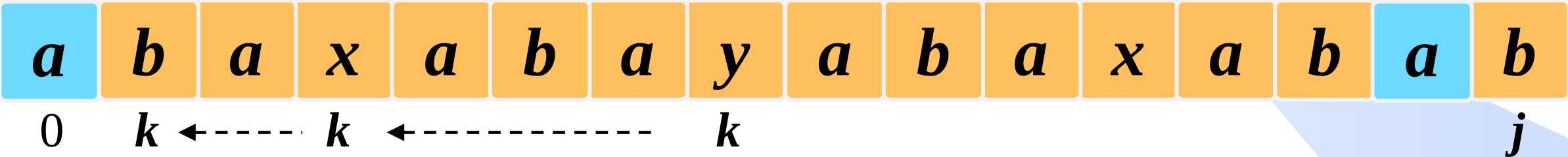
```
while(k >= 0 && p[k] != p[j])  
    k = next[k];  
next[j+1] = k+1;
```

已知 $next(j)=k$, 求 $next(j+1)$



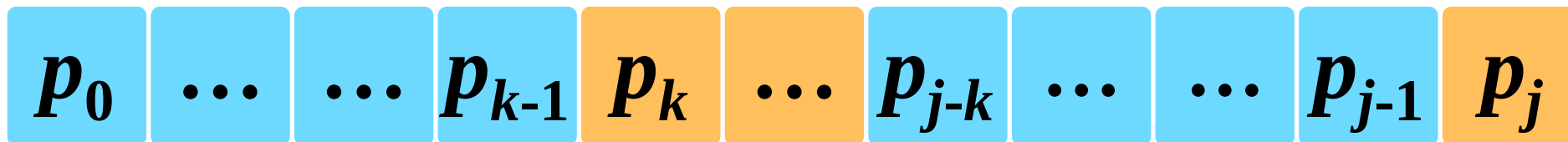
```
while(k >= 0 && p[k] != p[j])  
    k = next[k];  
next[j+1] = k+1;
```

已知 $next(j)=k$, 求 $next(j+1)$



```
while(k >= 0 && p[k] != p[j])  
    k = next[k];  
next[j+1] = k+1;
```

*next*数组的计算



```
void buildNext(char *P, int next[], int m){
    int k=next[0]=-1;
    for(int j=0; j<m-1; j++){ //求next[j+1], j+1<m
        while(k>=0 && P[k]!=P[j])
            k=next[k]; //求p0...pk-1的最长相等前后缀
        next[j+1]=++k;
    }
}
```


next数组的计算——时间复杂度分析

```
void buildNext(char *P, int next[], int m){
    int k=next[0]=-1;
    for(int j=0; j<m-1; j++){ //求next[j+1]
        while(k>=0 && P[k]!=P[j])
            k=next[k];
        next[j+1]=++k;
    }
}
```

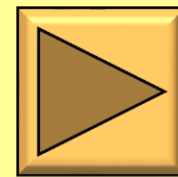
关键运算
字符比较

使k减小

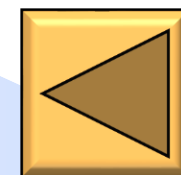
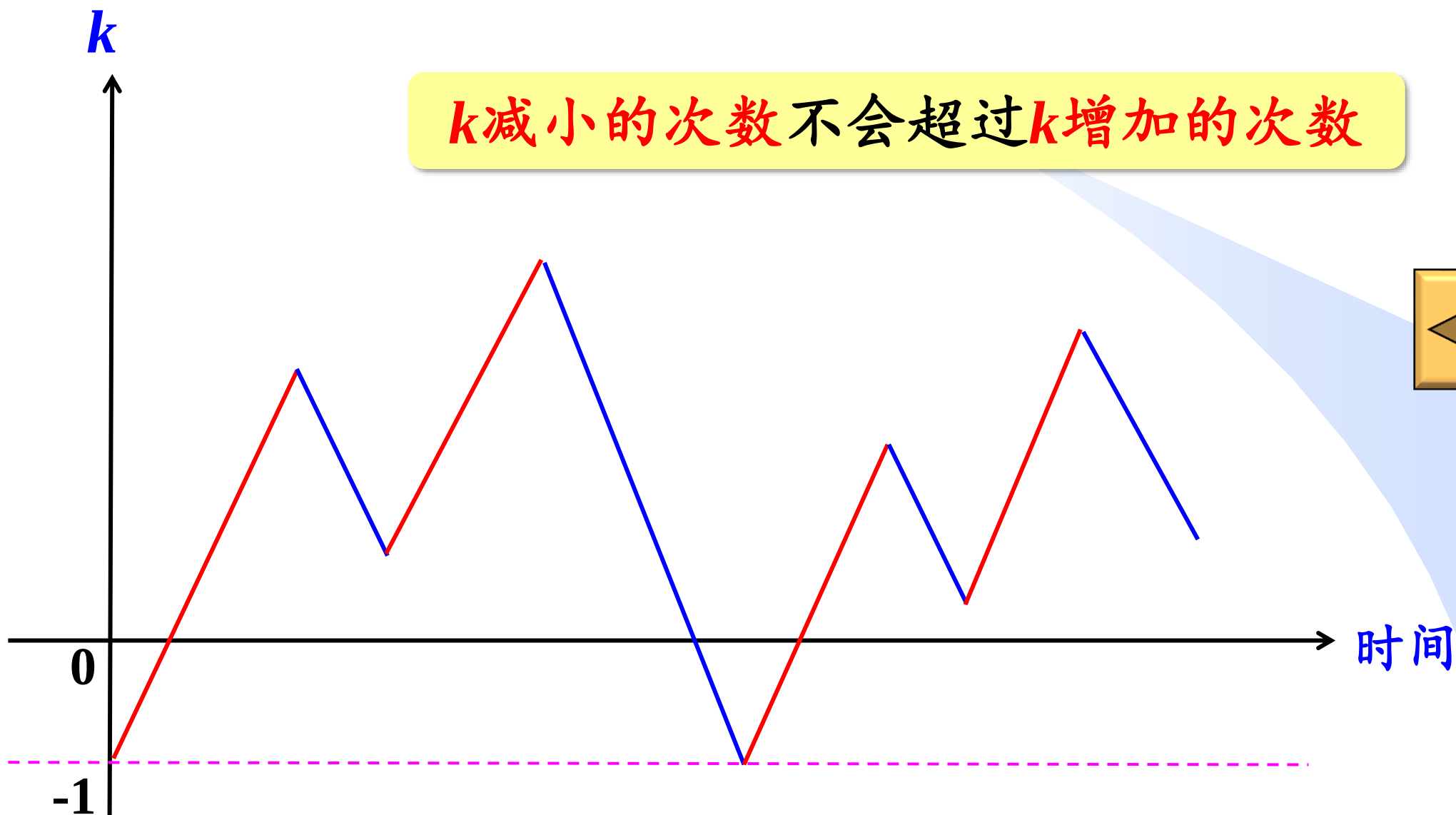
使k增加

执行次数取决于
k=next[k] 和
next[j+1]=++k
的次数

- 字符比较次数取决于k增加次数和k减小次数之和。
- ++k在for循环内, 执行次数不会超过m次, 即k增加不超过m次, 每次加1。
- k初值-1, 迭代过程中永远不会小于-1, 故k减小 ($k = \text{next}[k]$) 的次数不会超过k增加的次数 (即m次), 否则k就比-1小了。
- 故关键运算不会超过2m次, 即 $O(m)$



k 减小的次数不会超过 k 增加的次数



KMP算法——时间复杂度分析

关键运算
元素比较

B

```
int KMP(char *S, char *P, int n, int m){  
    int next[N], i=0, j=0;  
    buildNext(P, next, m); //O(m)  
    while(j<m && i<n)  
        if(j==-1 || S[i]==P[j]) i++, j++;  
        else j=next[j];  
    return (j==m)? i-m:-1;  
}
```

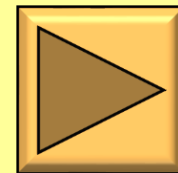
while循环迭代次数不仅
取决于*i*，还取决于*j*

使*j*减小

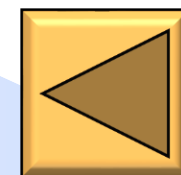
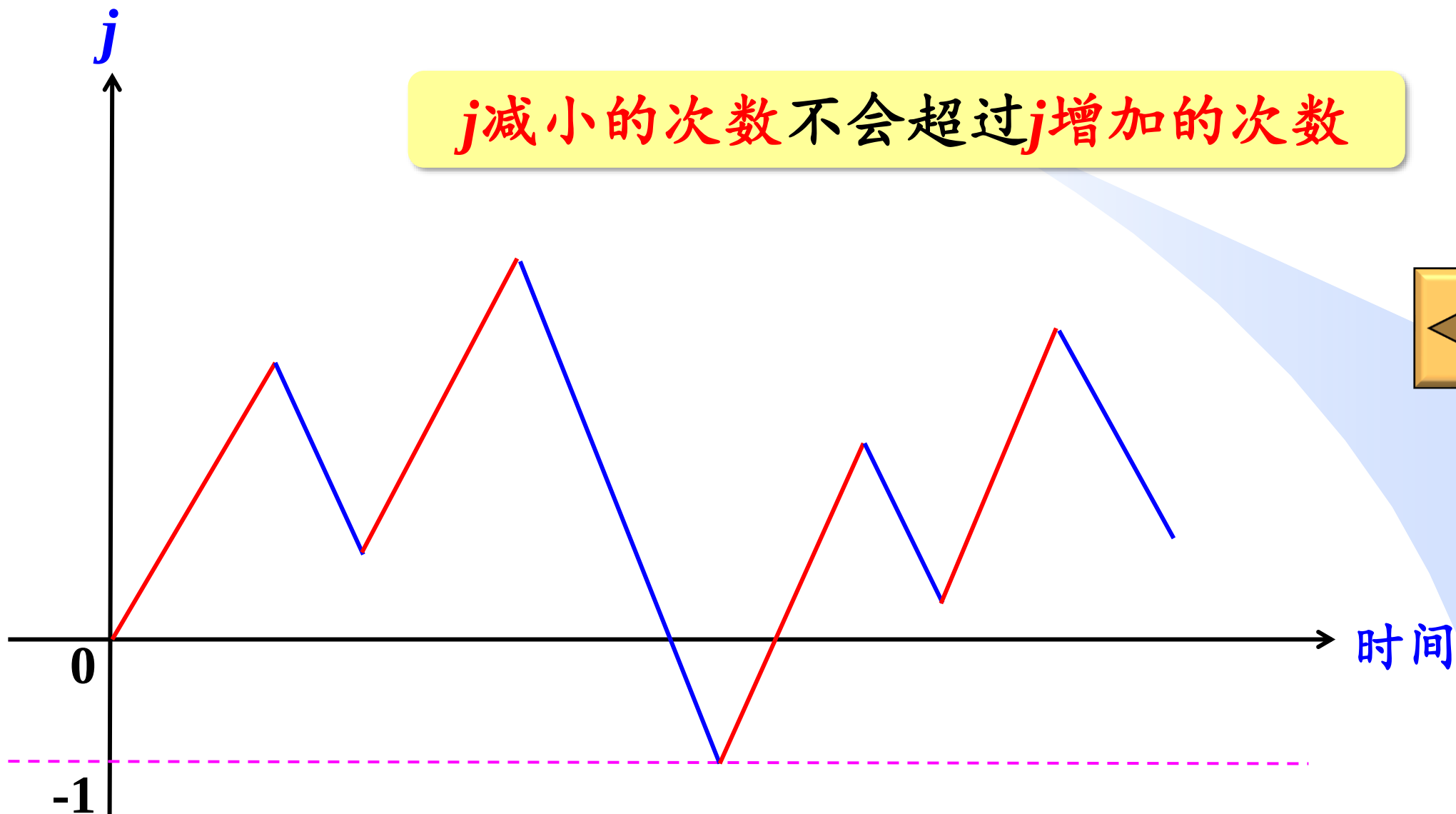
使*j*增加

KMP算法总时间复杂度 $O(n+m)$

- 扫描过程中*i*最多从0至*n*，*i*在循环内永远不减小，最多增加*n*次，每次加1，而*j++*与*i++*同步，故*j*也最多增加*n*次，每次加1。
- *j*最小是从-1开始增加，即*j++*的最小起点是-1，迭代过程中永远不会小于-1，故*j*减小（*j=next[j]*）的次数不会超过*j*增加的次数，即*n*次。
- 故关键运算不会超过 $2n$ 次，即 $O(n)$ 。



j 减小的次数不会超过 j 增加的次数



拓展

给定目标串 S 和模式串 P ，输出 P 在 S 中出现的次数，以及 P 在 S 中的所有匹配位置【吉林大学20级数据结构上机考试题】

```
int KMP(char S[], char P[], int n, int m){ //S长度n, P长度m
    int next[N], i=0, j=0, cnt=0; // cnt为P在S中出现次数
    buildNext(P, next, m); //计算next数组, 需要多算一位到next[m]
    while(j<m && i<n) {
        if(j==-1 || S[i]==P[j]) i++, j++;
        else j=next[j]; //确定P下次匹配位置
        if (j == m) { //匹配成功
            cnt++; printf("%d\n", i-m); //输出此时P在S中位置
            j=next[j]; //“假装”此处失配, 重新确定下次匹配位置
        }
    }
    return cnt;
}
```

KMP算法的隐藏应用举例

字符串长度为 n

- 最长相等的前后缀长度: $next[n]$.
- 第二长相等的后缀长度: $next[next[n]]$.
- 最长重复前缀: $next$ 数组中的最大值 $\max_i(next[i])$.

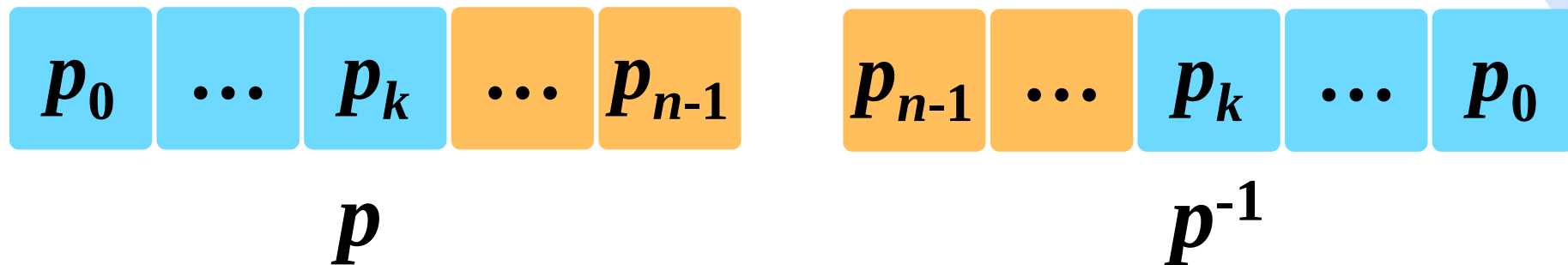
a b c a b c a b c x x x



KMP算法的隐藏应用举例——回文前缀

给定长度为 n 的字符串 p ，编写算法返回 p 的最长回文前缀的长度，如果 p 本身就是回文，则返回 p 的长度。【搜狗校园招聘笔试题】

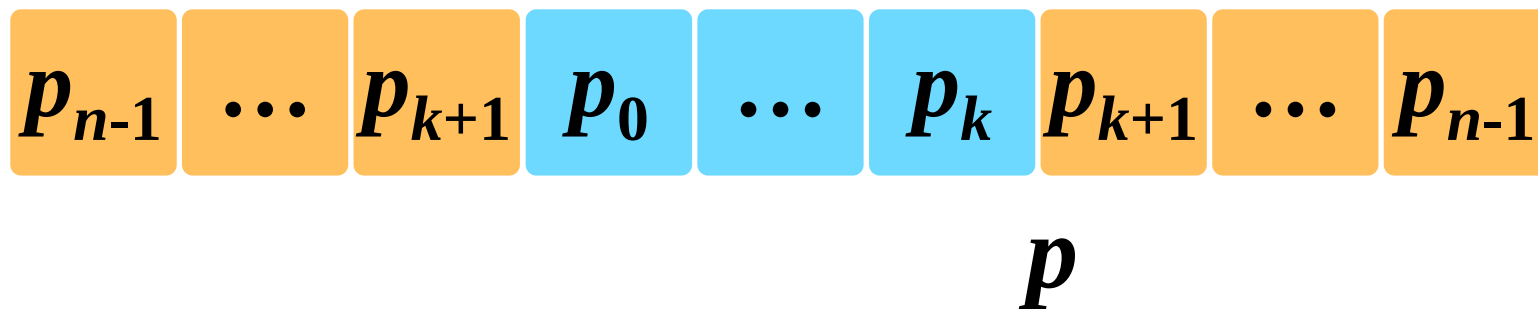
➤ 假定 p 的最长回文前缀为 $p_0 \dots p_k$ ，则有 $p_0 \dots p_k = p_k \dots p_0$



KMP算法的隐藏应用举例——回文前缀

给定长度为 n 的字符串 p ，在 p 前面最少添加几个字符可使 p 变为回文串？并输出该最短回文串。【字节跳动、华为、腾讯、爱奇艺、微软、谷歌面试题[LeetCode214](#)】

➤ 假定 p 的最长回文前缀为 $p_0 \dots p_k$



KMP算法隐藏应用举例——循环节

如果一个字符串 S 可由它的一个最小子串 P 重复 k 次构成，则 P 称为 S 的循环节， k 称为循环周期。例如：

$S = a b c a b c a b c a b c$

循环节（最小重复单元）为 $a b c$ ，循环周期为4.

KMP算法隐藏应用举例——循环节

给定一个非空字符串 S ，检查它可否可以通过由它的一个子串重复多次构成。【腾讯、字节跳动、快手、谷歌、携程、招商银行、浦发银行面试题[LeetCode459](#)】

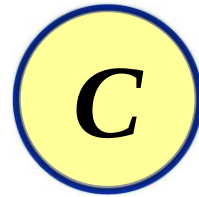
例如：

$S = a b c a b c a b c a b c$ ，输出True

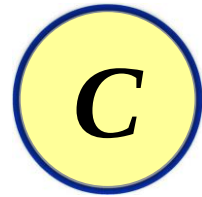
$S = a b a$ ，输出False



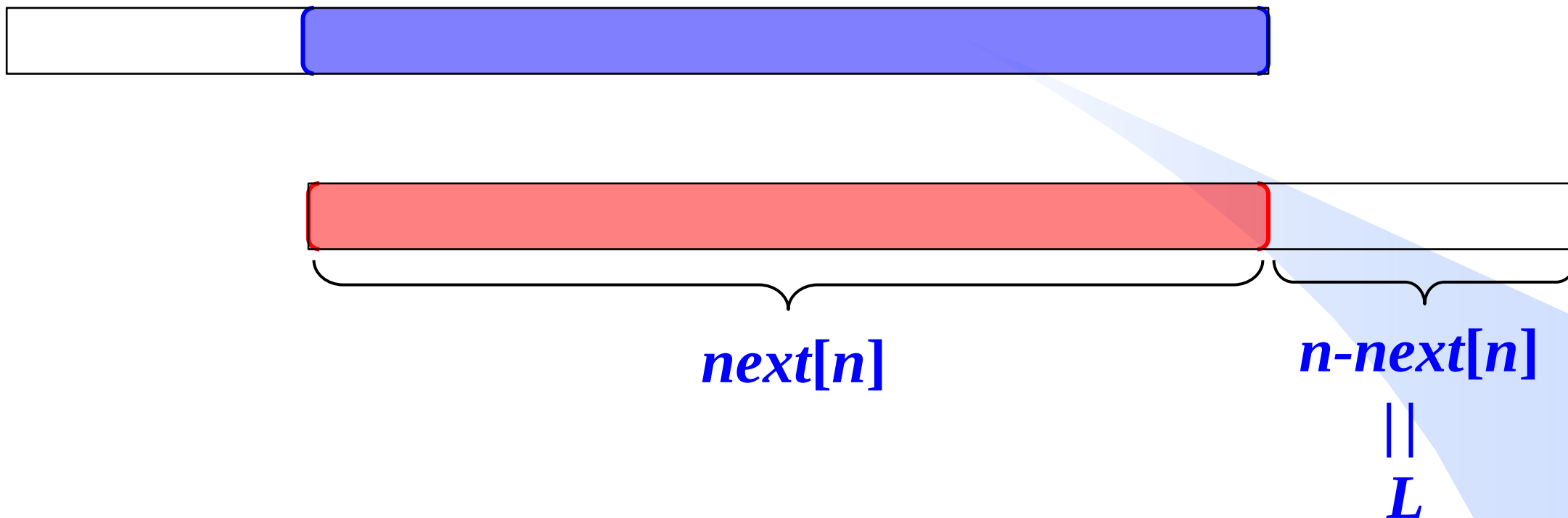
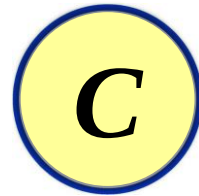
KMP算法隐藏应用举例——循环节



KMP算法隐藏应用举例——循环节

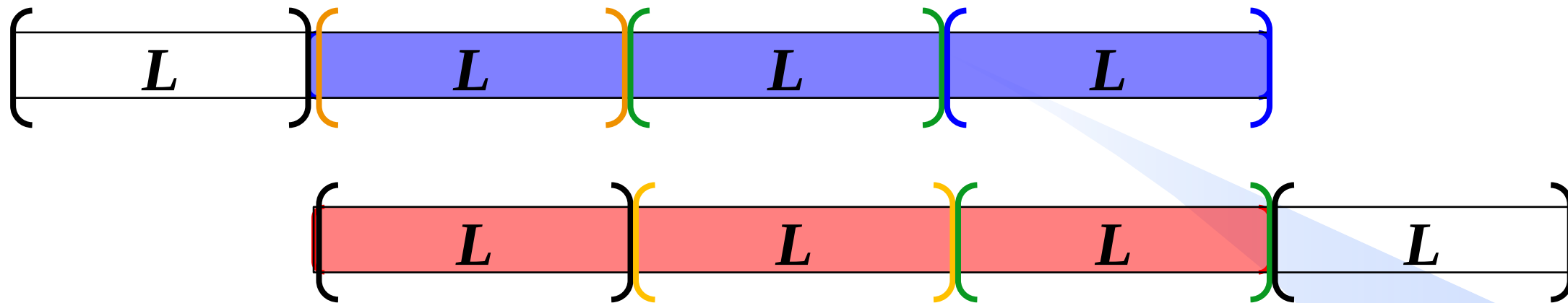


KMP算法隐藏应用举例——循环节



KMP算法隐藏应用举例——循环节

C



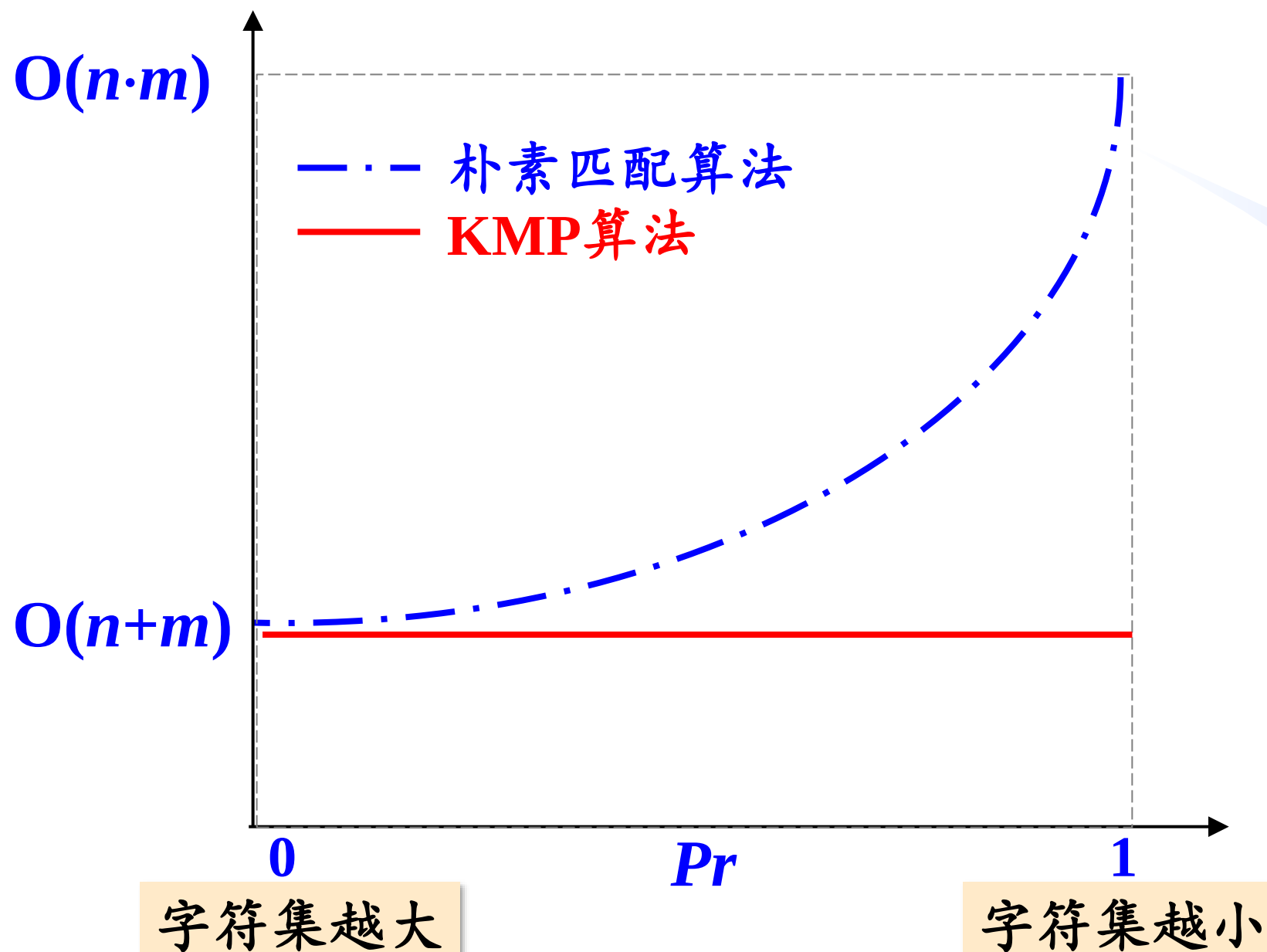
- 如果 $next[n] > 0$ 且 $n \% L == 0$, 则 S 可由循环节完全循环构成:
- 循环节长度为: $L = n - next[n]$, 循环周期为: n / L

KMP算法隐藏应用举例——循环节

- 如果 $next[n] > 0$ 且 $n \% L == 0$ ，则 S 可由循环节完全循环构成：
- 循环节长度为： $L = n - next[n]$ ，循环周期为： n / L

```
const int N=1e4+10;
bool repeatedSubstringPattern(char *s){
    int next[N], n=strlen(s);
    buildNext(s,next,n); //next数组要多算一位，算到next[n]
    int L=n-next[n];
    if(next[n]>0 && n%L==0) return true;
    return false;
}
```

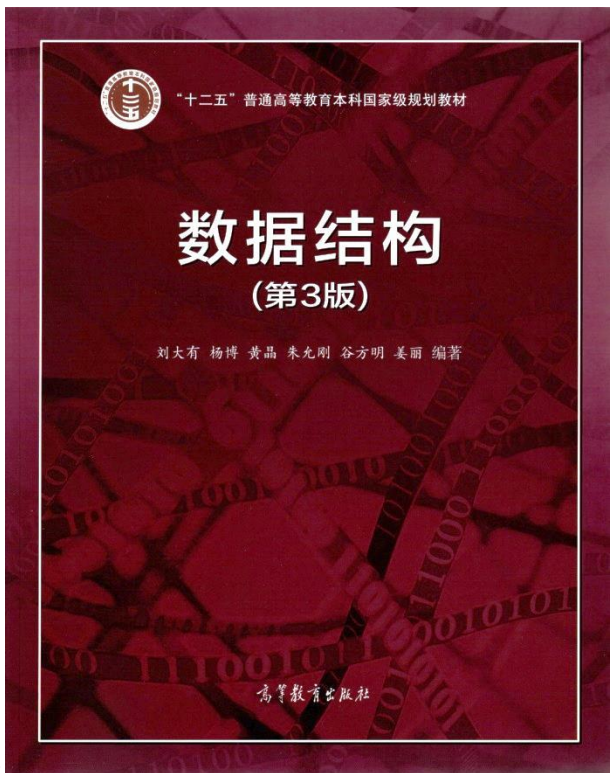
总结



字符集越小，字符
单次成功比对概率
 Pr 越高，KMP 算
法相对于朴素匹配
算法优势越大

$$Pr = 1/k$$

k 为字符集大小

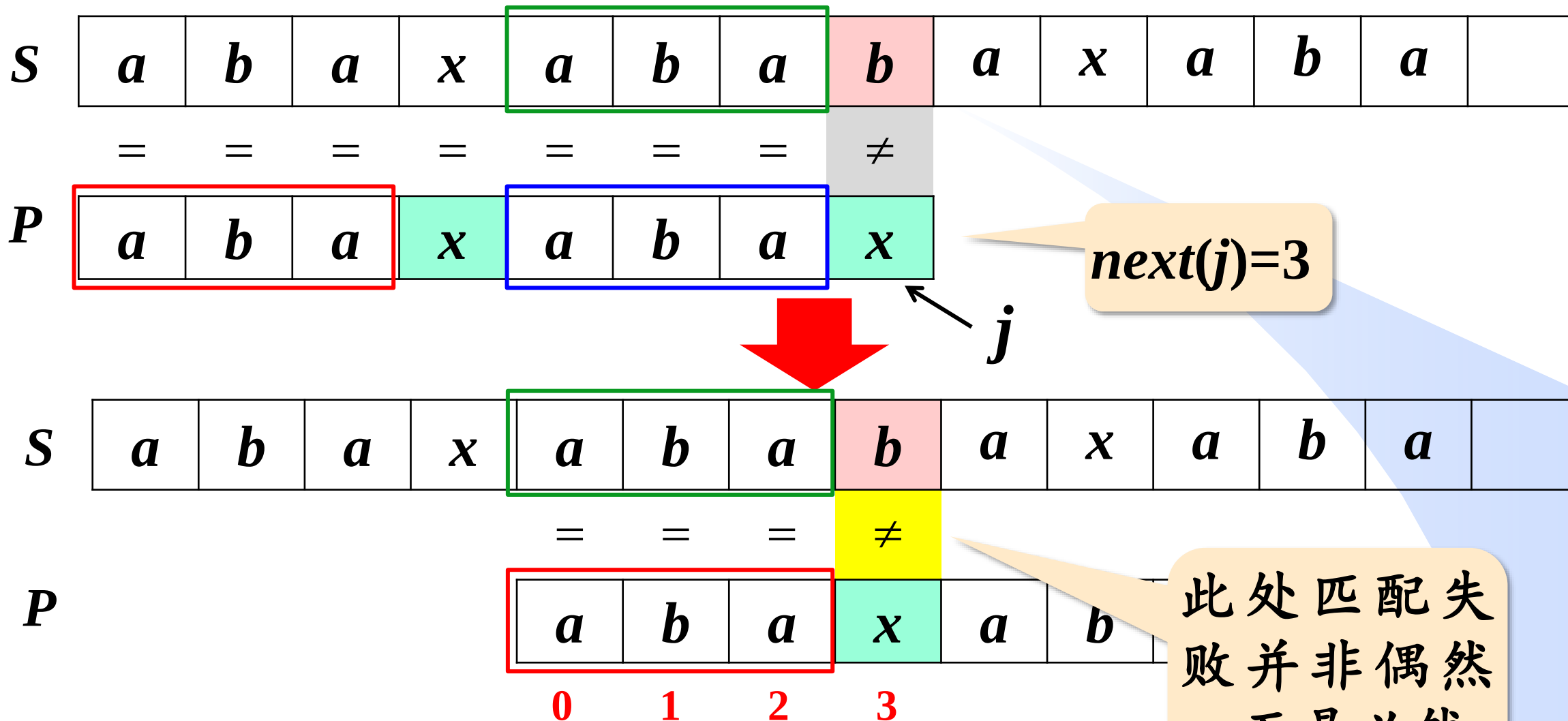


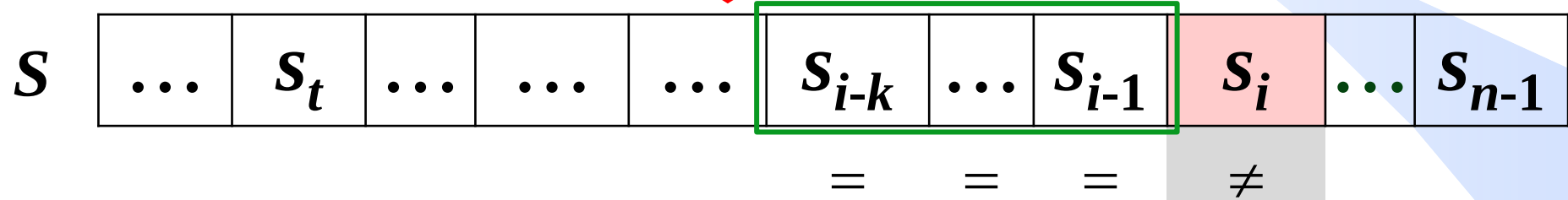
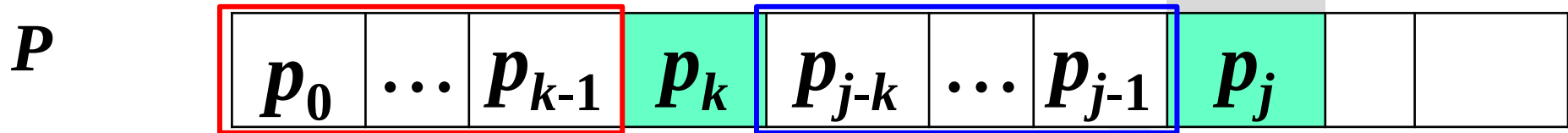
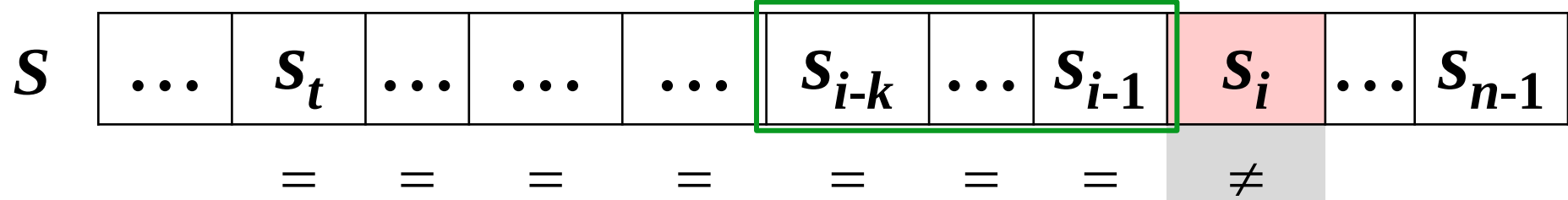
字符串模式匹配

- 模式匹配基本概念
- 朴素模式匹配算法
- KMP算法
- **KMP算法的改进**

数据之法
结构之美
算法之道

原始next函数的不足





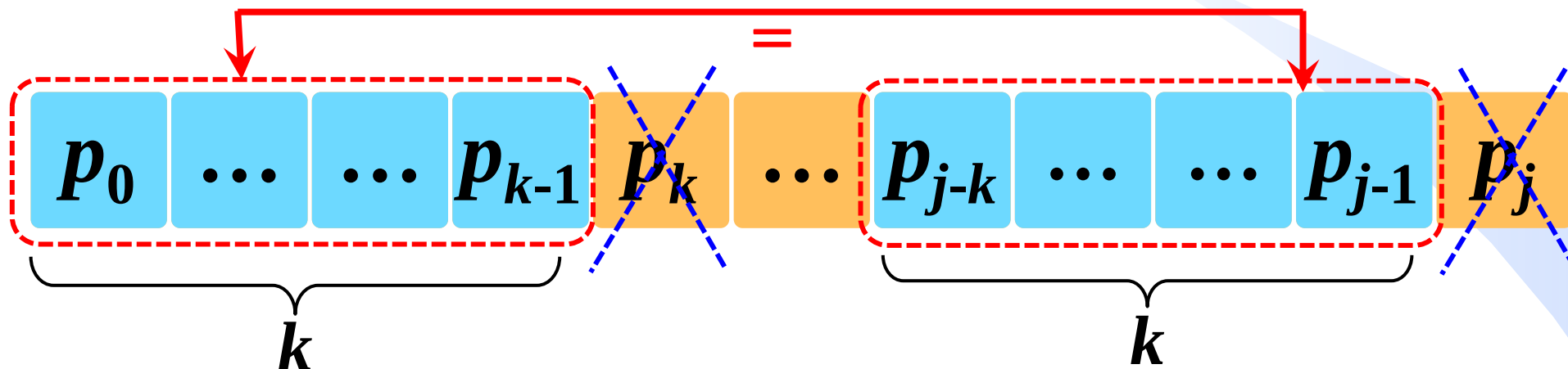
在 p_j 处失配，必有 $s_i \neq p_j$ ，

➤ 如果 $p_j = p_k$ ，则必有 $s_i \neq p_k$ ，下趟匹配必然失败；

➤ 改进方法：在确定下次匹配位置时，除了找失配位置前的最长相等前后缀之外，还应比较其后面的两个字符（即前缀后面的字符 p_k 和当前失配位置的字符 p_j ），确保 $p_j \neq p_k$

改进的 $next(j)$ 函数

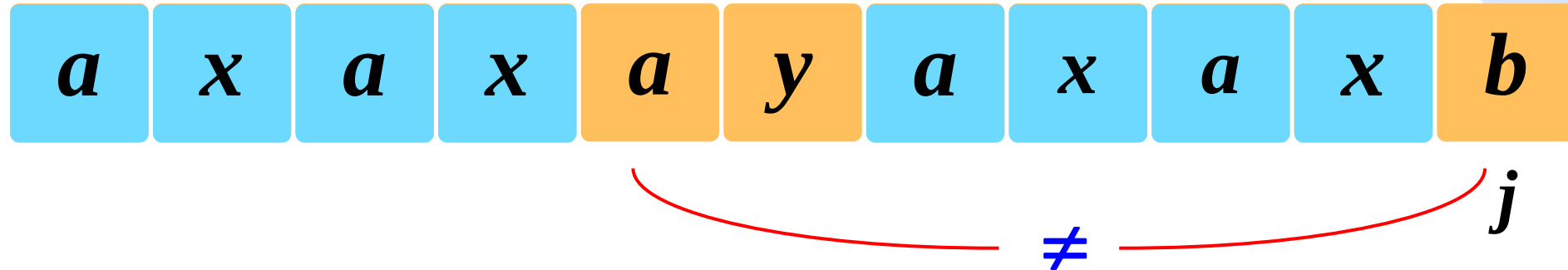
额外要求：最长相等前后缀后面的两个字符（即前缀后面的字符 p_k 和当前位置 j 的字符 p_j ）不能相等，即 $p_j \neq p_k$



$$nextval(j) = \begin{cases} -1, & j = 0 \\ \max\{k \mid p_0 \dots p_{k-1} = p_{j-k} \dots p_{j-1} \text{ 且 } p_k \neq p_j, 0 < k < j\}, & \text{存在 } k \\ 0, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀但 } p_0 \neq p_j \\ -1, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀且 } p_0 = p_j \end{cases}$$

例1

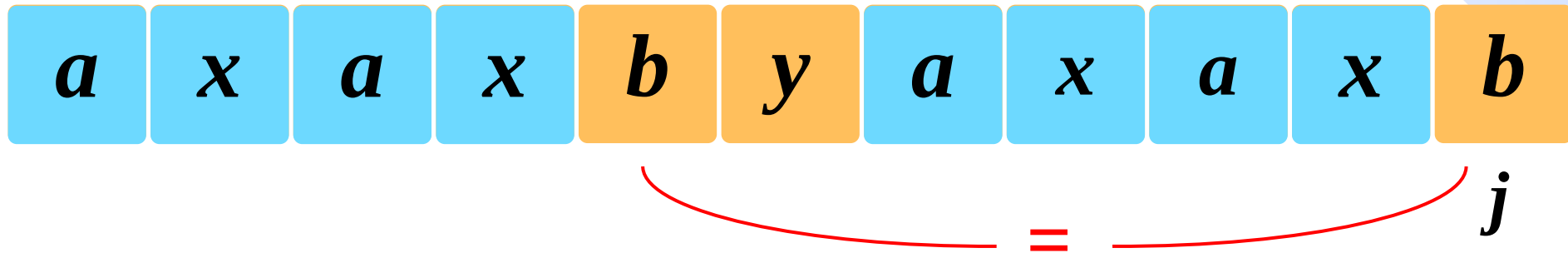
$$nextval(j) = \begin{cases} -1, & j = 0 \\ \max\{k | p_0 \dots p_{k-1} = p_{j-k} \dots p_{j-1} \text{ 且 } p_k \neq p_j, 0 < k < j\}, & \text{存在 } k \\ 0, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀但 } p_0 \neq p_j \\ -1, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀且 } p_0 = p_j \end{cases}$$



$$nextval(j) = 4$$

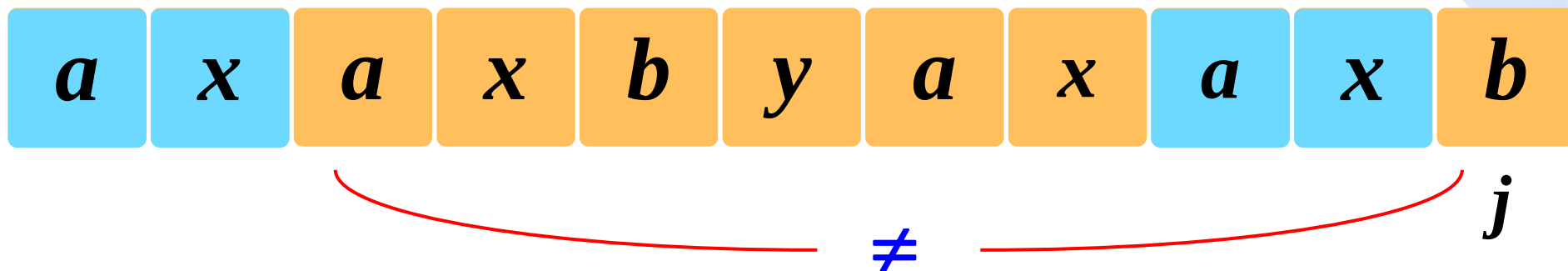
例2

$$nextval(j) = \begin{cases} -1, & j = 0 \\ \max\{k | p_0 \dots p_{k-1} = p_{j-k} \dots p_{j-1} \text{ 且 } p_k \neq p_j, 0 < k < j\}, & \text{存在 } k \\ 0, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀但 } p_0 \neq p_j \\ -1, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀且 } p_0 = p_j \end{cases}$$



例2 (续)

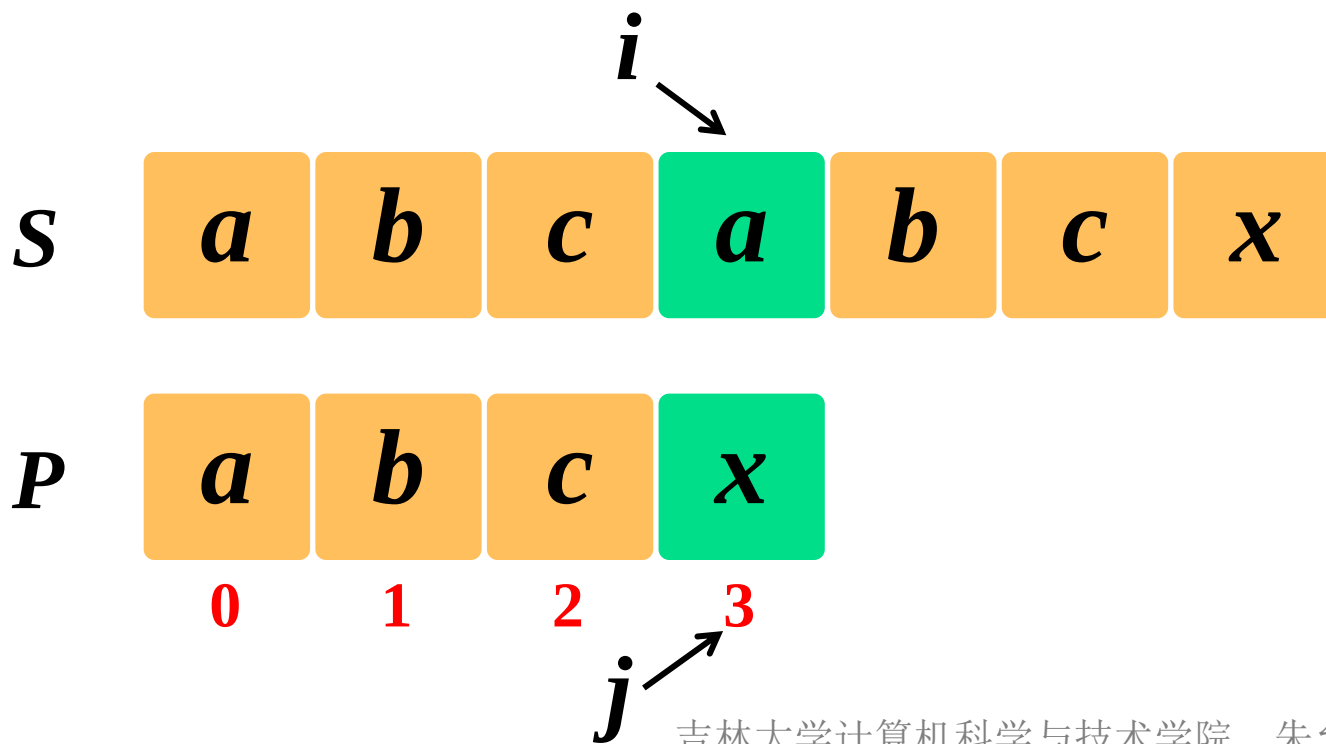
$$nextval(j) = \begin{cases} -1, & j = 0 \\ \max\{k | p_0 \dots p_{k-1} = p_{j-k} \dots p_{j-1} \text{ 且 } p_k \neq p_j, 0 < k < j\}, & \text{存在 } k \\ 0, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀但 } p_0 \neq p_j \\ -1, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀且 } p_0 = p_j \end{cases}$$



$$nextval(j) = 2$$

改进的 $next(j)$ 函数

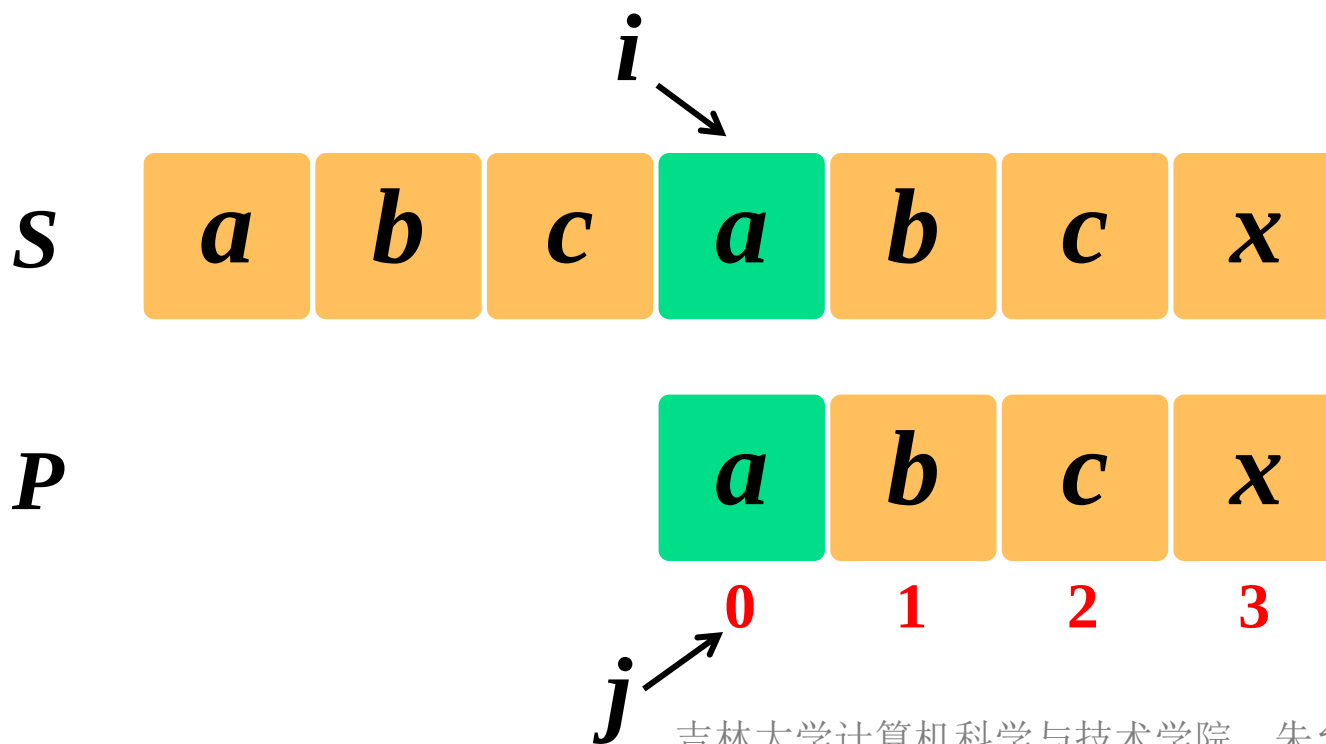
$$nextval(j) = \begin{cases} -1, & j = 0 \\ \max\{k | p_0 \dots p_{k-1} = p_{j-k} \dots p_{j-1} \text{ 且 } p_k \neq p_j, 0 < k < j\}, & \text{存在 } k \\ \mathbf{0}, & \mathbf{p_0 \dots p_{j-1} \text{ 不存在相等的前后缀但 } p_0 \neq p_j} \\ -1, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀且 } p_0 = p_j \end{cases}$$



至少可以将 P_0 移动到失配位置，与 S 进行下一次比对

改进的 $next(j)$ 函数

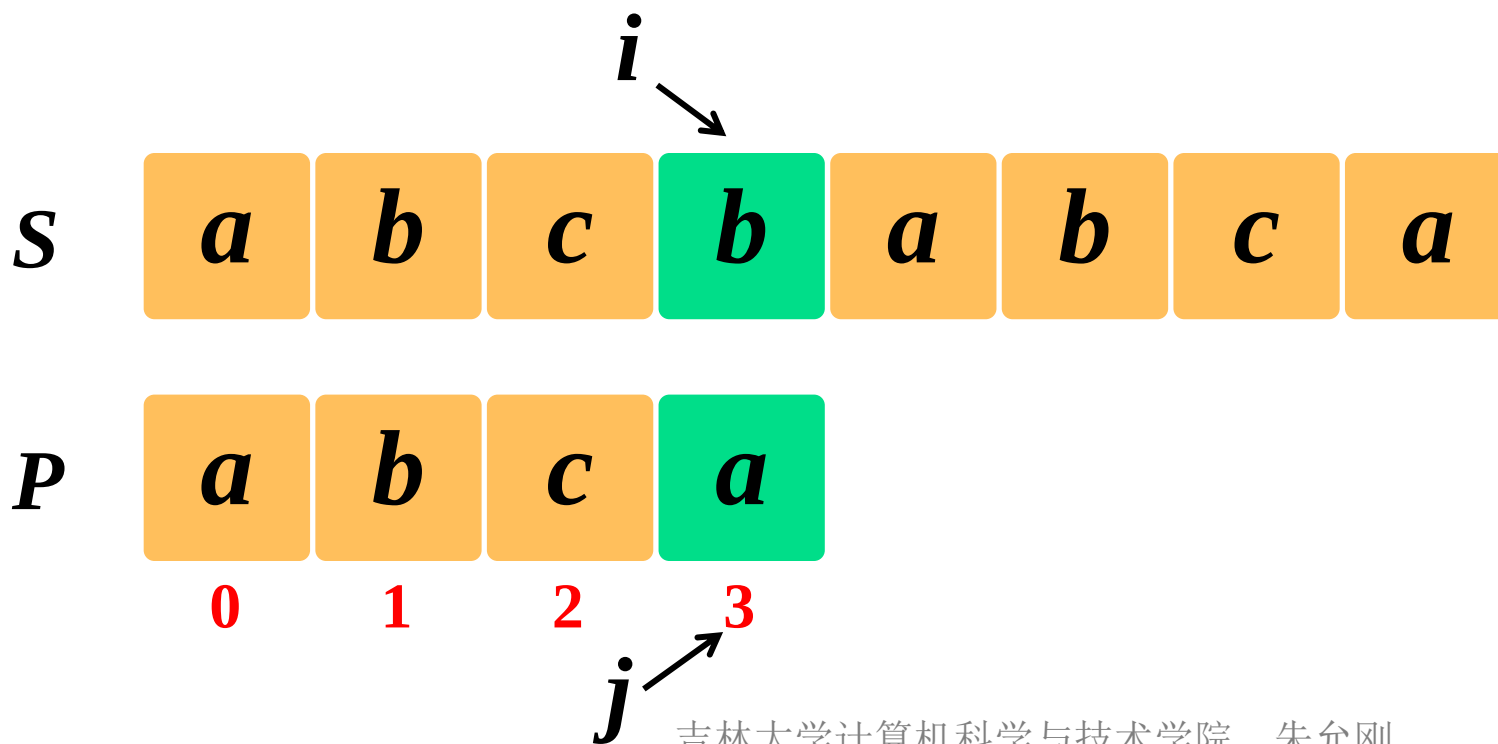
$$nextval(j) = \begin{cases} -1, & j = 0 \\ \max\{k | p_0 \dots p_{k-1} = p_{j-k} \dots p_{j-1} \text{ 且 } p_k \neq p_j, 0 < k < j\}, & \text{存在 } k \\ \mathbf{0}, & \mathbf{p_0 \dots p_{j-1} \text{ 不存在相等的前后缀但 } p_0 \neq p_j} \\ -1, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀且 } p_0 = p_j \end{cases}$$



至少可以将 P_0 移动到失配位置，与 S 进行下一次比对

改进的 $next(j)$ 函数

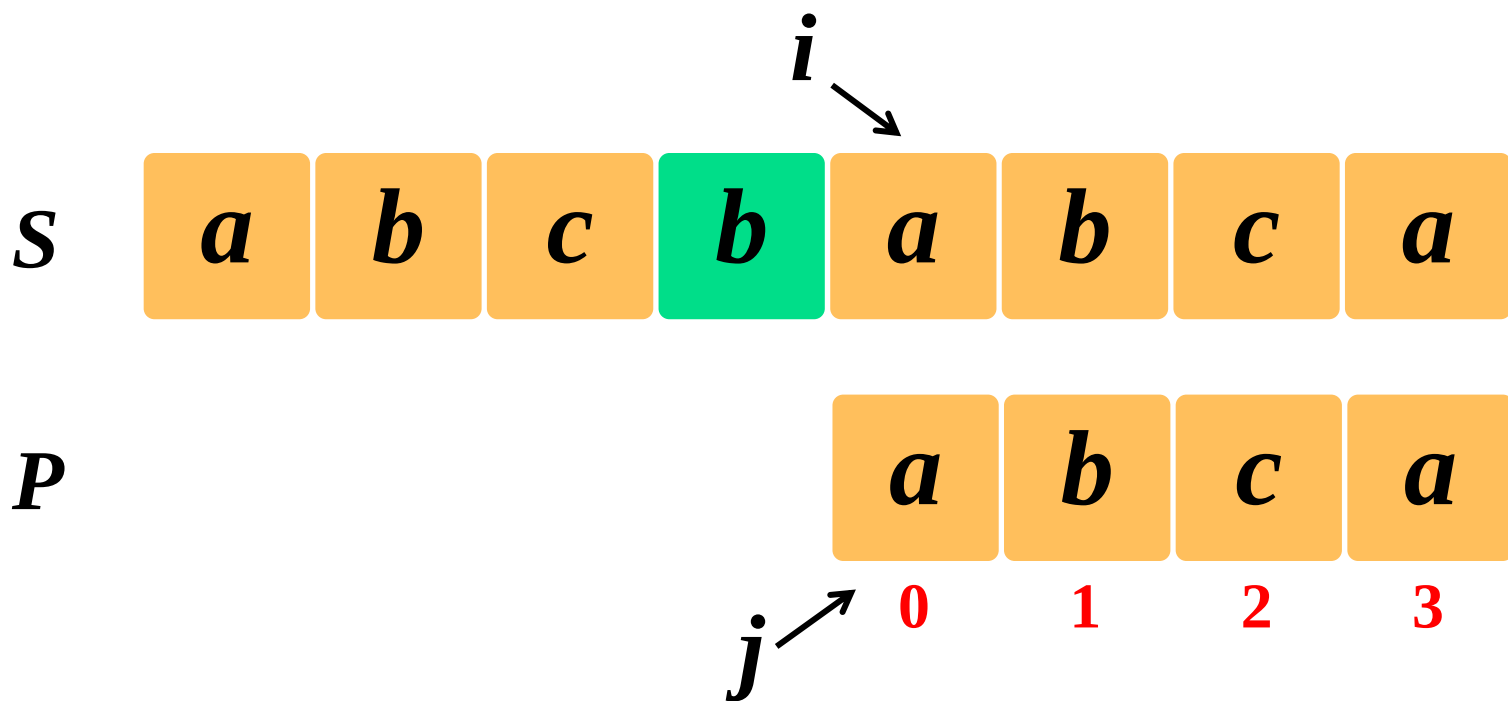
$$nextval(j) = \begin{cases} -1, & j = 0 \\ \max\{k | p_0 \dots p_{k-1} = p_{j-k} \dots p_{j-1} \text{ 且 } p_k \neq p_j, 0 < k < j\}, & \text{存在 } k \\ 0, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀但 } p_0 \neq p_j \\ -1, & \text{p}_0 \dots \text{p}_{j-1} \text{ 不存在相等的前后缀且 } p_0 = p_j \end{cases}$$



P_j 前所有位置都不可能与 S 匹配，应将 P 整体移过失配位置

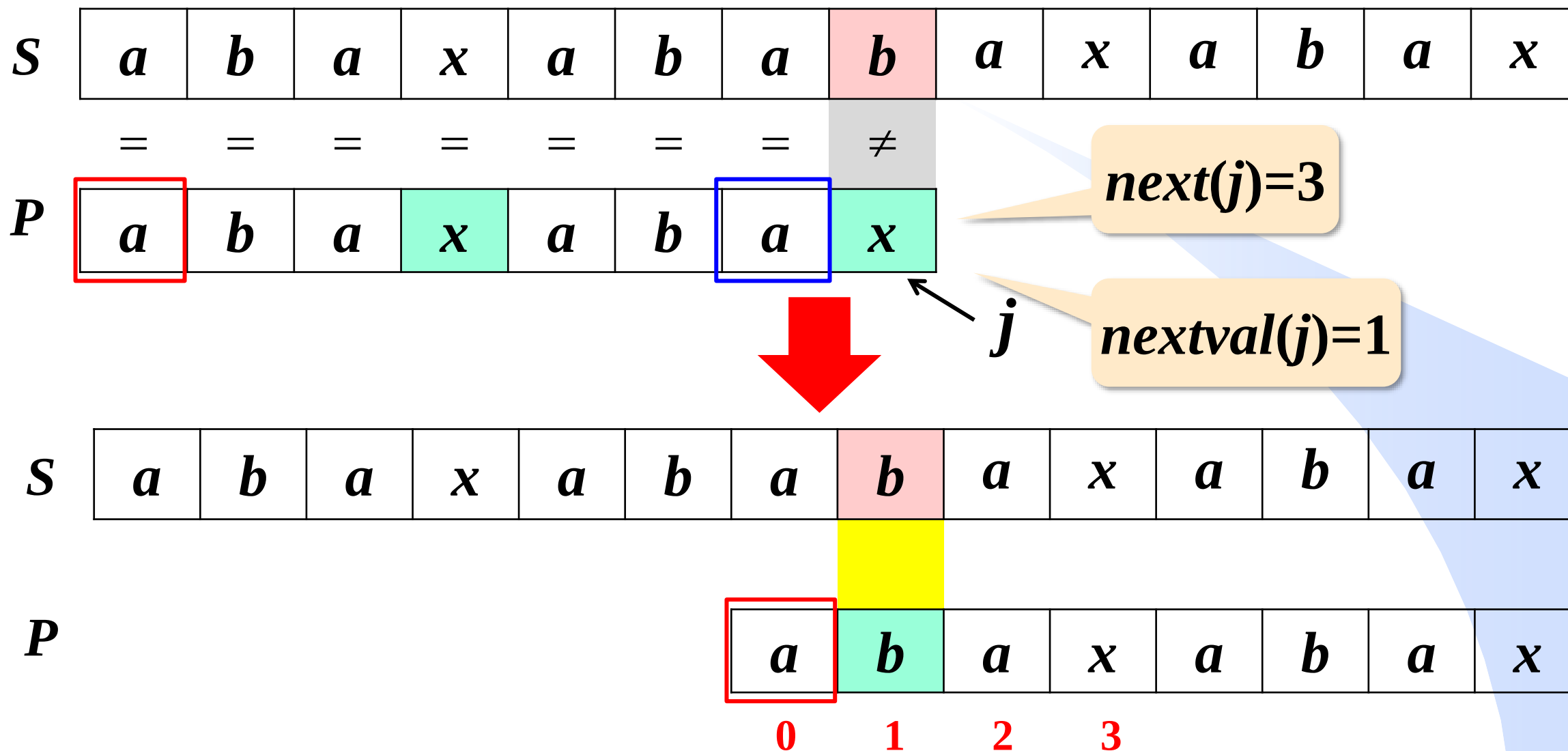
改进的 $next(j)$ 函数

$$nextval(j) = \begin{cases} -1, & j = 0 \\ \max\{k | p_0 \dots p_{k-1} = p_{j-k} \dots p_{j-1} \text{ 且 } p_k \neq p_j, 0 < k < j\}, & \text{存在 } k \\ 0, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀但 } p_0 \neq p_j \\ -1, & p_0 \dots p_{j-1} \text{ 不存在相等的前后缀且 } p_0 = p_j \end{cases}$$



P_j 前所有位置都不可能与 S 匹配，应将 P 整体移过失配位置

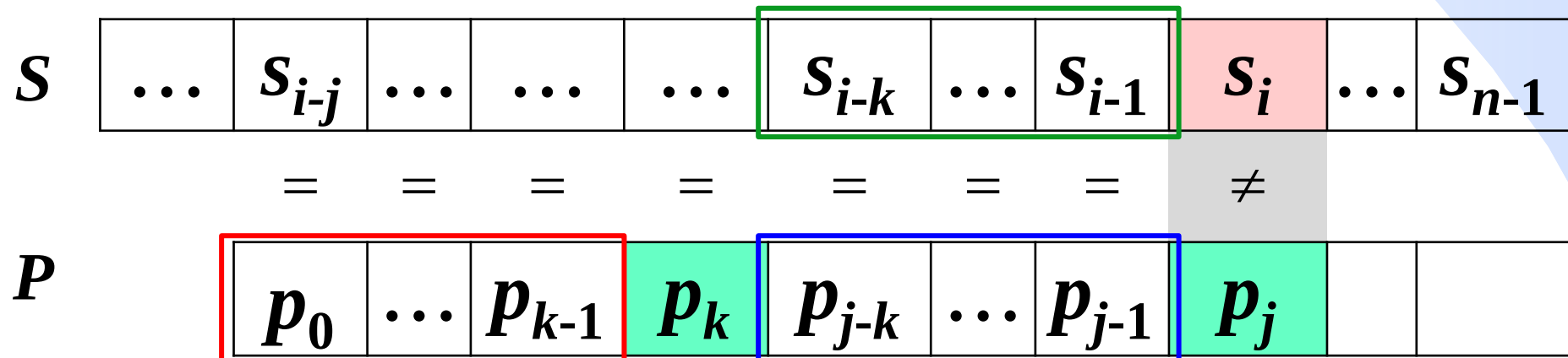
回顾之前的例子——改进的 $next$ 函数



启示

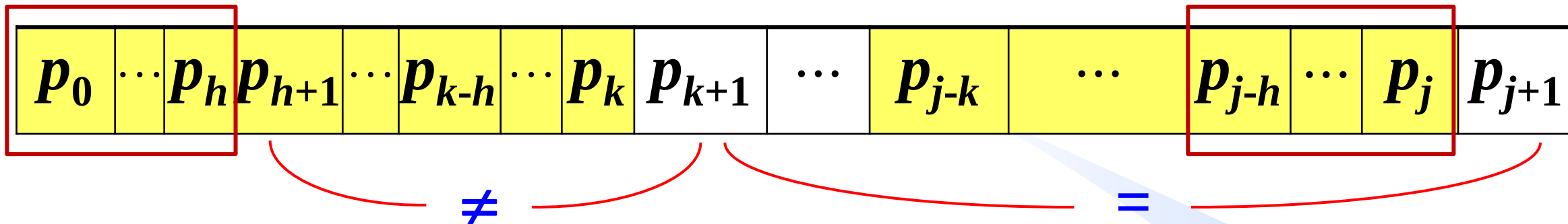
更加充分地利用以往比对所提供的信息

- *next*: 以往成功比对所获得的“经验”—— $S_{i-j} \dots S_{i-1}$ 是什么
- *nextval*: 以往失败比对所吸取的“教训”—— S_i 不是什么



课下思考: *nextval*函数的实现

C



```
void buildNextval(char P[], int nextval[], int m) {  
    int k = nextval[0] = -1;  
    for (int j = 0; j < m; j++) { //求nextval[j+1]  
        while (k >= 0 && P[k] != P[j])  
            k = nextval[k];  
        if(P[j+1] != P[k+1]) nextval[j+1] = ++k;  
        else nextval[j+1] = nextval[++k];  
    }  
}
```